

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 2-2: Regularization and Optimization

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Aug/27/2026



Administration

CSCI 598H Course Overview



Weeks 2-4

Weeks 5-11

Weeks 12-16

Module 1

Deep Learning Basics

- **Linear Classifier and Optimization**
- CNNs and BackPropagation
- RNNs, Attention, Transformers

Module 2

Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

Module 3

Engineering Systems

- Robotics
- Earth science
- Medical imaging

Recap Linear Classification

Softmax Normalization + Cross-Entropy Loss + Regularization

Image Classification: A core task in Computer Vision



[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

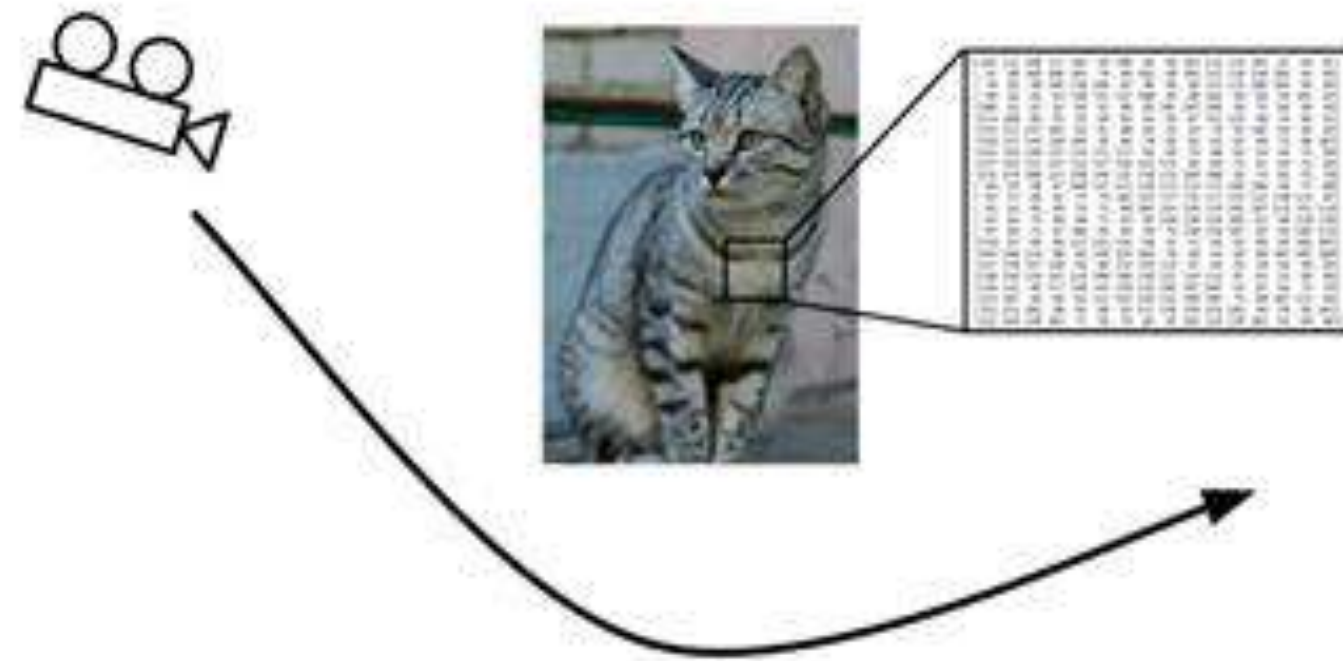
(assume given a set of labels)
{dog, cat, truck, plane, ...}



cat
dog
bird
deer
truck

Recall from last time: Challenges of recognition

Viewpoint



Illumination



[This image](#) is [CC0 1.0](#) public domain

Deformation



[This image](#) by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)

Occlusion



[This image](#) by [jonsson](#) is licensed under [CC-BY 2.0](#)

Clutter



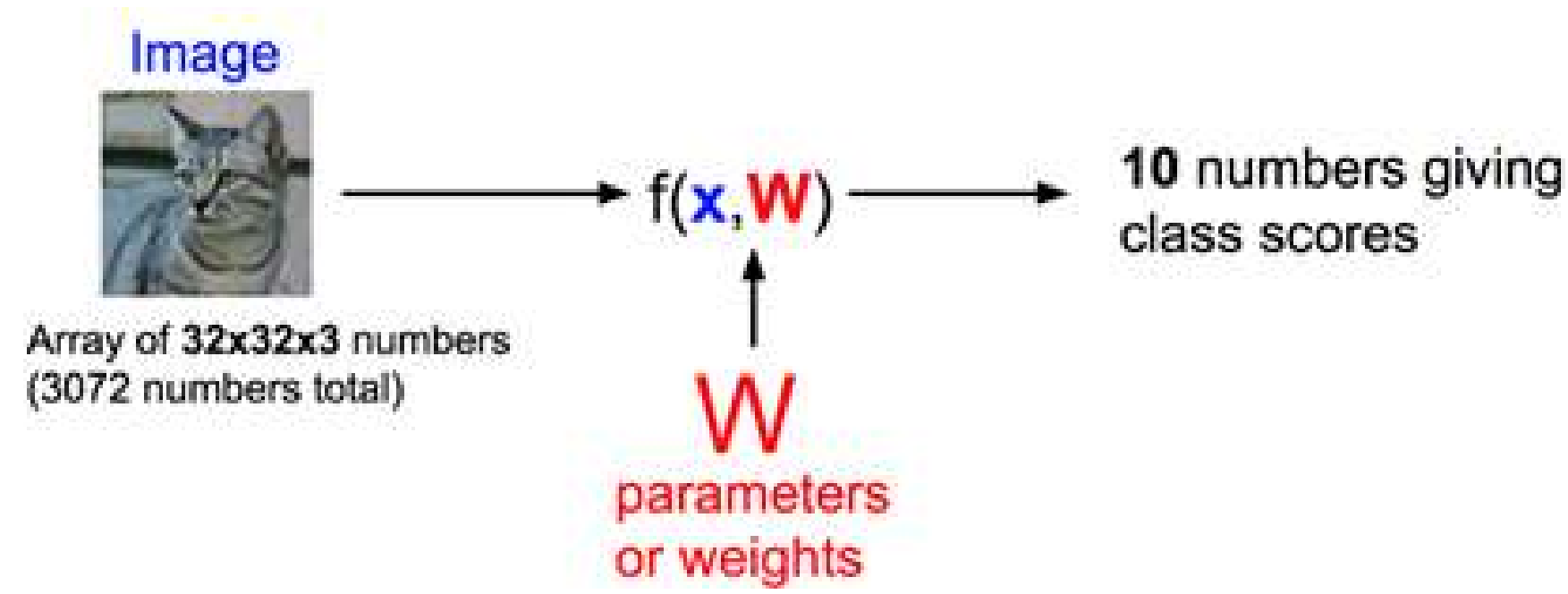
[This image](#) is [CC0 1.0](#) public domain

Intraclass Variation



[This image](#) is [CC0 1.0](#) public domain

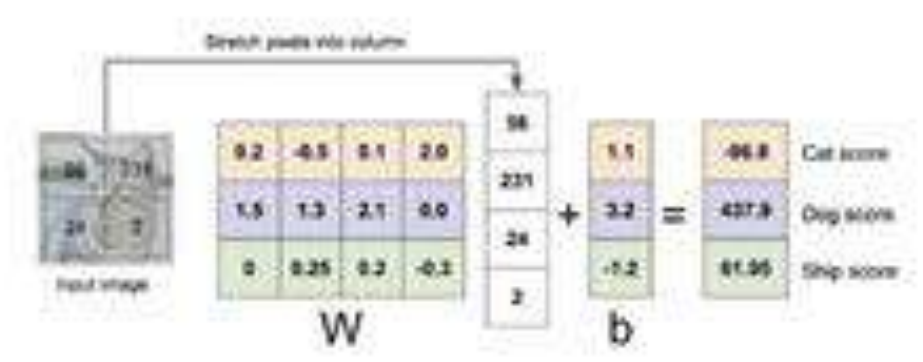
Recall from last time: Linear Classifier



$$f(x, W) = Wx + b$$

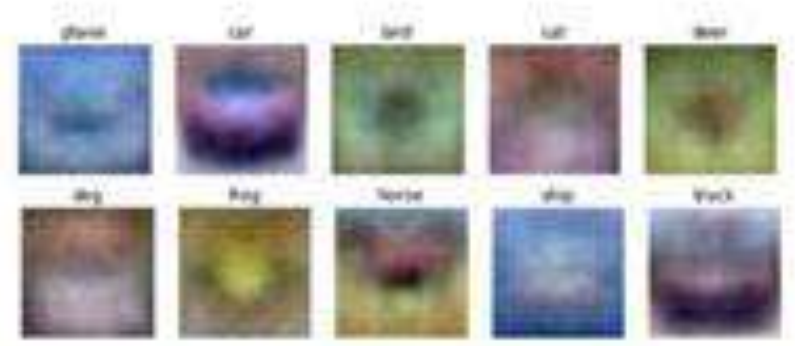
Algebraic Viewpoint

$$f(x, W) = Wx$$



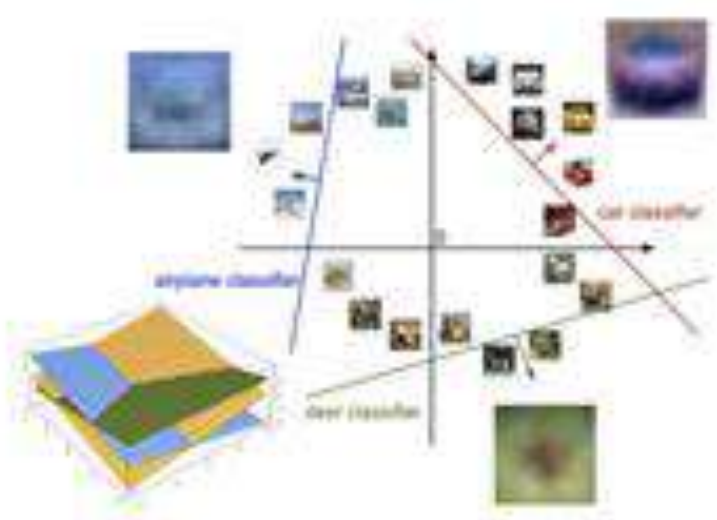
Visual Viewpoint

One template per class



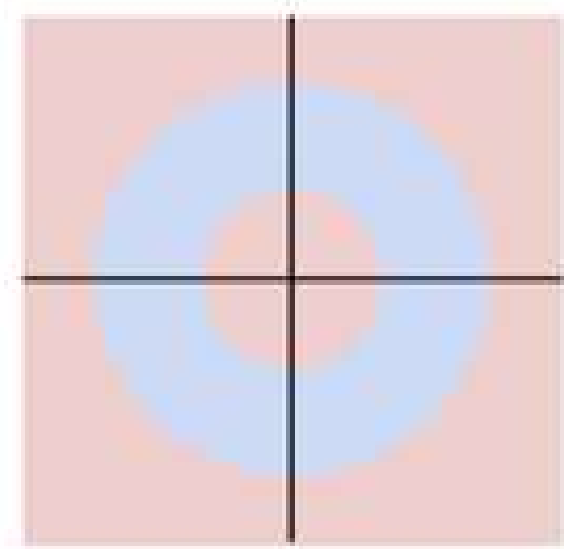
Geometric Viewpoint

Hyperplanes cutting up space



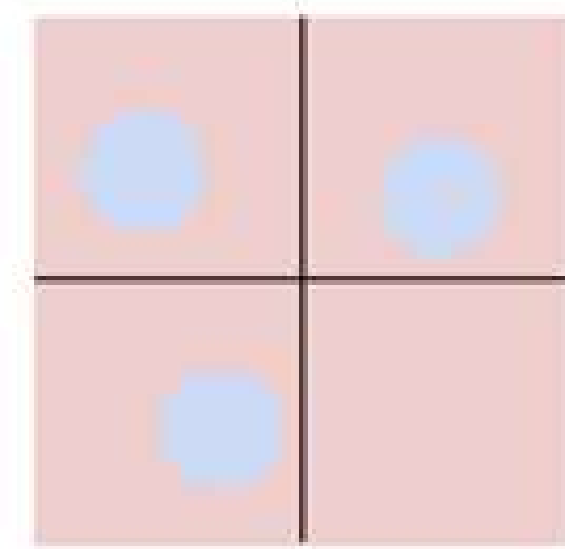
Class 1:
 $1 \leq L2 \text{ norm} \leq 2$

Class 2:
Everything else



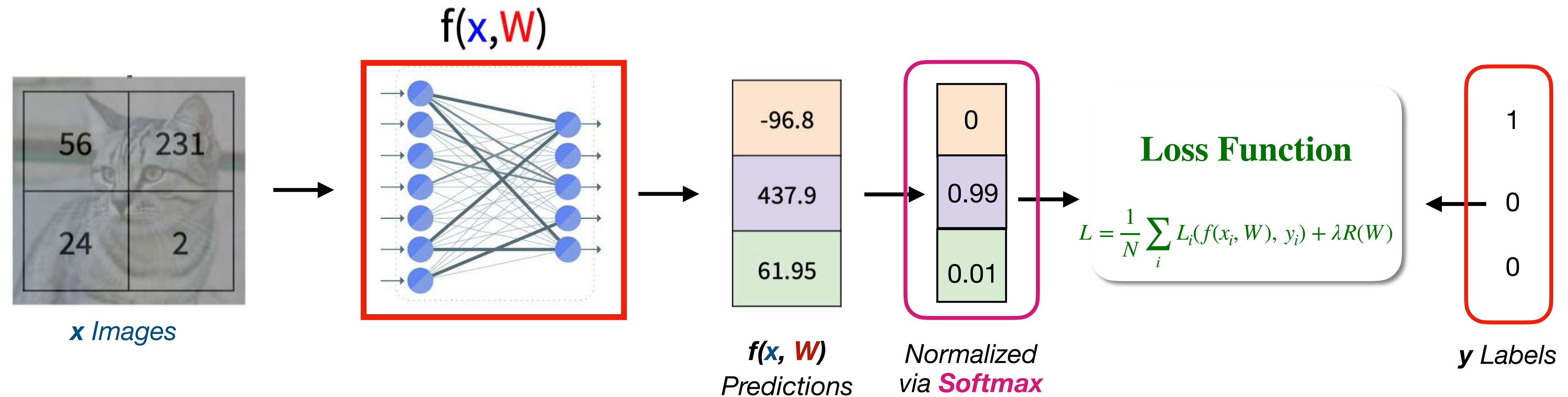
Class 1:
Three modes

Class 2:
Everything else



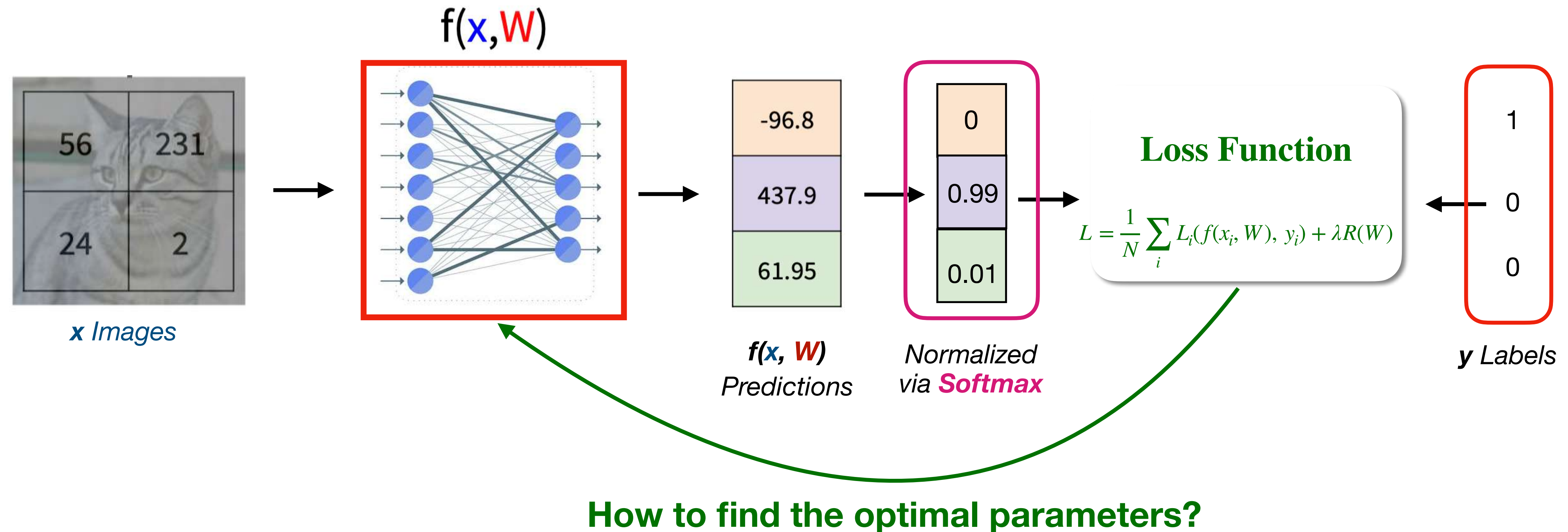
Linear Classifier

Model Training: Softmax Normalization + Cross-Entropy Loss + Regularization



Linear Classifier

Model Training: Softmax Normalization + Cross-Entropy Loss + Regularization



Optimization

- Random Descent
- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Adam
- Learning rate

https://d2l.ai/chapter_optimization/index.html

Additional materials and code demo





Goal: Find the *optimal path* to the bottom of the mountain.

Strategy #1: A first but very bad idea solution: Random Search

**Random
Search**



Strategy #1: A first but very bad idea solution: Random Search

**Random
Search**



Strategy #1: A first but very bad idea solution: Random Search



```
start_w = np.array([-5.0, 4.0])

def random_search(start_w, num_trials=30, search_scale=6.0):
    best_w = start_w.astype(float).copy()
    best_loss = loss_fn(best_w)
    history = []

    history.append(
        {
            "step": 0,
            "candidate_w": best_w.copy(),
            "candidate_loss": best_loss,
            "best_w": best_w.copy(),
            "best_loss": best_loss,
        }
    )

    for step in range(1, num_trials):
        candidate_w = np.random.uniform(-search_scale, search_scale, size=2)
        candidate_loss = loss_fn(candidate_w)

        if candidate_loss < best_loss:
            best_loss = candidate_loss
            best_w = candidate_w.copy()

        history.append(
            {
                "step": step,
                "candidate_w": candidate_w.copy(),
                "candidate_loss": candidate_loss,
                "best_w": best_w.copy(),
                "best_loss": best_loss,
            }
        )

    return best_w, best_loss, history
```

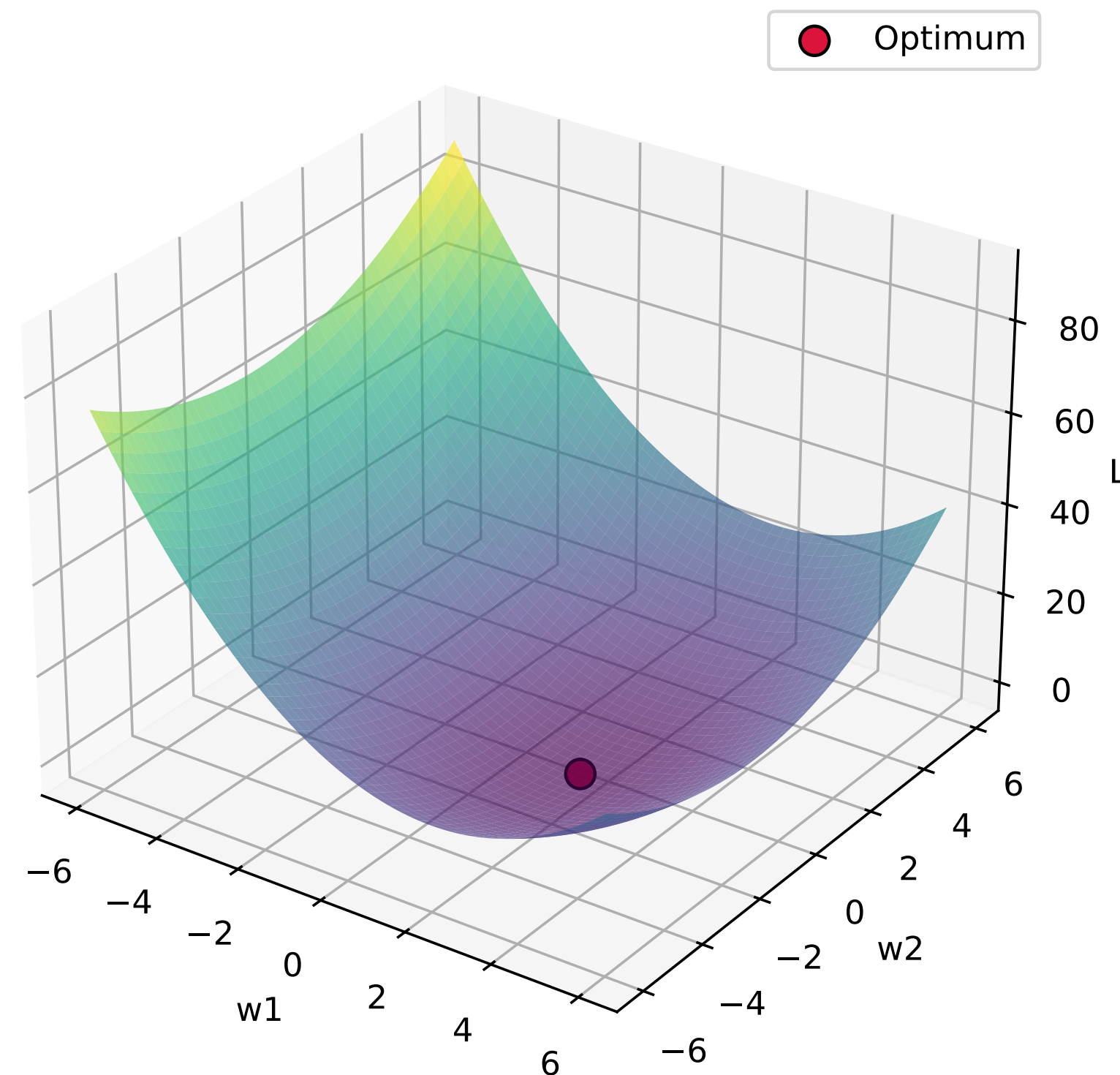
Toy example for Understanding

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

Q1: What is the optimal solution?

$$[w_1, w_2] = [2, -1]$$



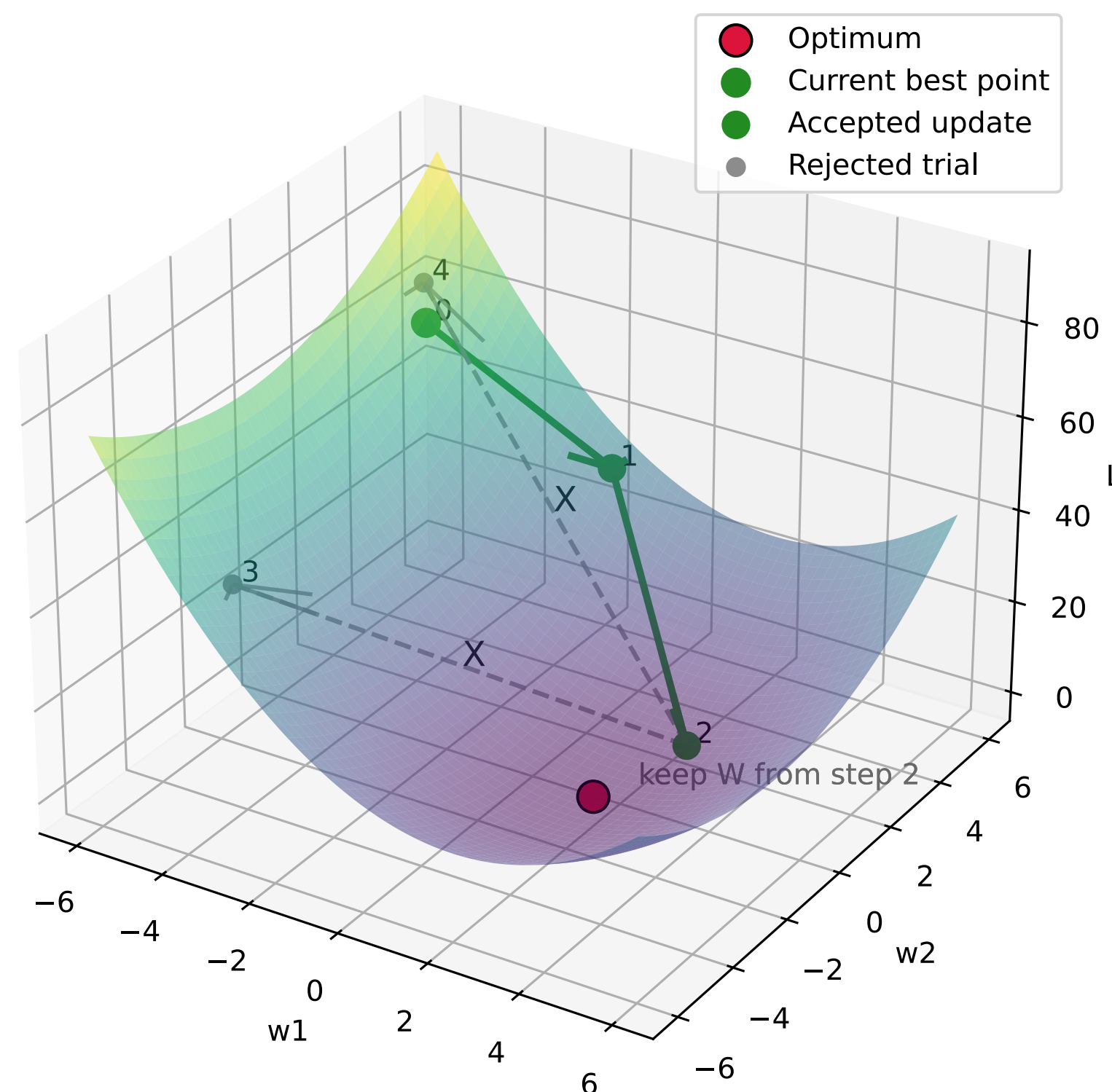
Lets see how well Random Search works on the toy example ...

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

Q1: What is the optimal solution?

$[w_1, w_2] = [2, -1]$



Random search result

```
start_w  = [-5.00, 4.00]
```

```
best_w   = [2.21, -0.72]
```

```
best_loss = 0.0841
```

```
step 0: candidate_w = [-5.00, 4.00], candidate_loss = 61.5000, best_loss = 61.5000
```

```
step 1: candidate_w = [-1.51, 5.41], candidate_loss = 32.8236, best_loss = 32.8236
```

```
step 2: candidate_w = [2.78, 1.18], candidate_loss = 2.9993, best_loss = 2.9993
```

```
step 3: candidate_w = [-4.13, -4.13], candidate_loss = 42.4420, best_loss = 2.9993
```

```
step 4: candidate_w = [-5.30, 4.39], candidate_loss = 67.8820, best_loss = 2.9993
```

```
step 5: candidate_w = [1.21, 2.50], candidate_loss = 6.7328, best_loss = 2.9993
```

```
step 6: candidate_w = [-5.75, 5.64], candidate_loss = 82.1464, best_loss = 2.9993
```

```
step 7: candidate_w = [3.99, -3.45], candidate_loss = 6.9633, best_loss = 2.9993
```

```
step 8: candidate_w = [-3.82, -3.80], candidate_loss = 37.7679, best_loss = 2.9993
```

```
step 9: candidate_w = [-2.35, 0.30], candidate_loss = 19.7558, best_loss = 2.9993
```

Can we do better than Random Search?

If you were hiking downhill, how would you choose your next step?

Why not random search?

- Many sampled points are **rejected**
- Computation is still wasted
- Slow convergence

```
Random search result
start_w  = [-5.00, 4.00]
best_w   = [2.21, -0.72]
best_loss = 0.0841
step 0: candidate_w = [-5.00, 4.00], candidate_loss = 61.5000, best_loss = 61.5000
step 1: candidate_w = [-1.51, 5.41], candidate_loss = 32.8236, best_loss = 32.8236
step 2: candidate_w = [2.78, 1.18], candidate_loss = 2.9993, best_loss = 2.9993
step 3: candidate_w = [-4.13, -4.13], candidate_loss = 42.4420, best_loss = 2.9993
step 4: candidate_w = [-5.30, 4.39], candidate_loss = 67.8820, best_loss = 2.9993
step 5: candidate_w = [1.21, 2.50], candidate_loss = 6.7328, best_loss = 2.9993
step 6: candidate_w = [-5.75, 5.64], candidate_loss = 82.1464, best_loss = 2.9993
step 7: candidate_w = [3.99, -3.45], candidate_loss = 6.9633, best_loss = 2.9993
step 8: candidate_w = [-3.82, -3.80], candidate_loss = 37.7679, best_loss = 2.9993
step 9: candidate_w = [-2.35, 0.30], candidate_loss = 19.7558, best_loss = 2.9993
```

Strategy #2: Follow the slope

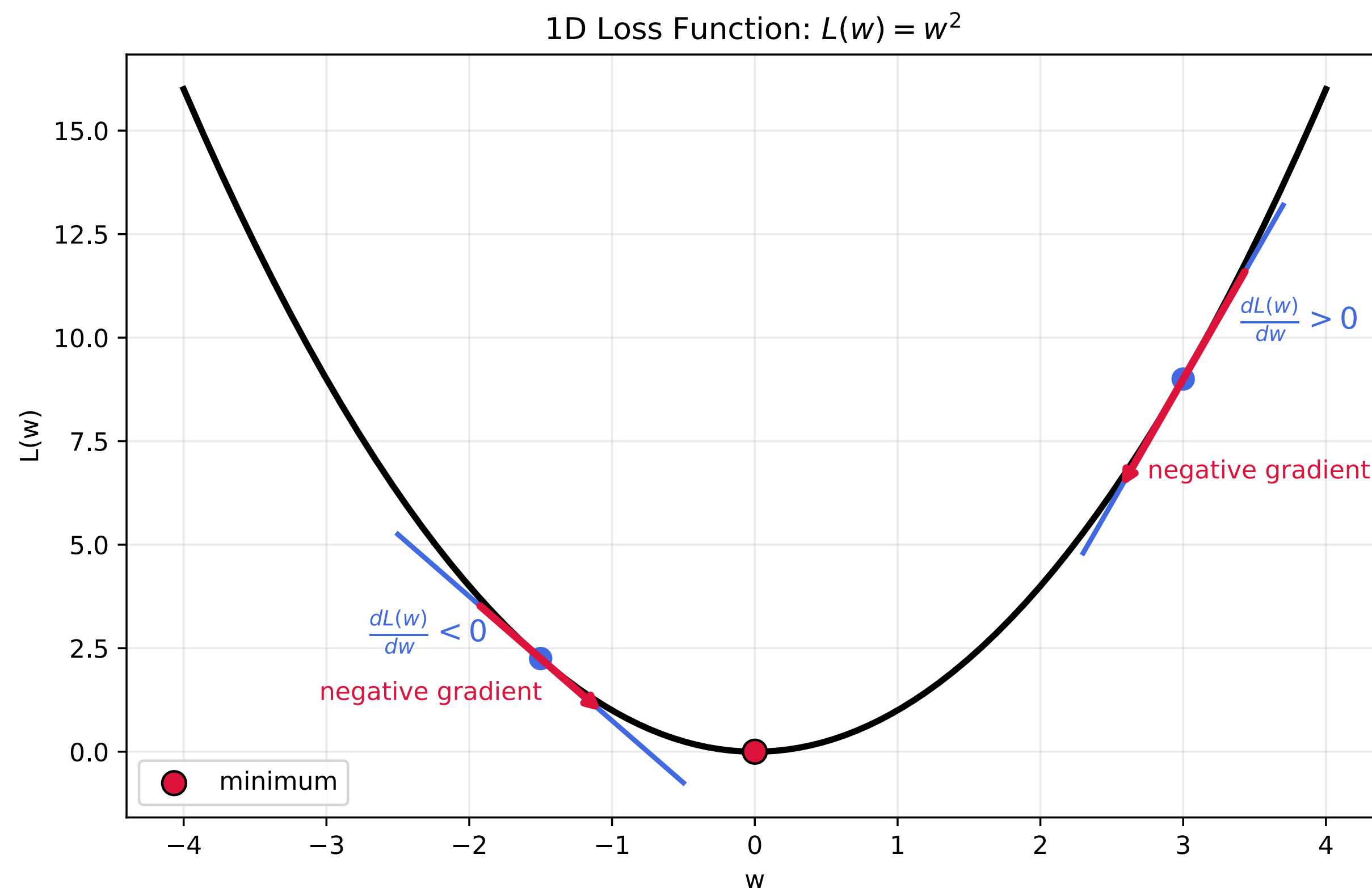


Strategy #2: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

h : a small perturbation



Example:

$$L(w) = w^2 \quad \frac{dL(w)}{dw} = 2w$$

- The direction of steepest descent is **the negative gradient!**
- The gradient descent with a learning rate η :

$$w = w - \eta \frac{dL(w)}{dw}$$

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

h : a small perturbation

Q2: What are (partial) derivatives in 2-dimension?

What about higher dimensions, i.e. n-dimension?

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h} \quad \boxed{h: \text{a small perturbation}}$$

In 2-dimension, the partial derivatives of a function along with w_1 and w_2 :

$$\frac{\partial L(w_1, w_2)}{\partial w_1} = \lim_{h_1 \rightarrow 0} \frac{L(w_1 + h_1, w_2) - L(w_1, w_2)}{h_1}$$

$$\frac{\partial L(w_1, w_2)}{\partial w_2} = \lim_{h_2 \rightarrow 0} \frac{L(w_1, w_2 + h_2) - L(w_1, w_2)}{h_2}$$

So the **gradient** is the vector of partial derivatives:

$$\nabla L(w_1, w_2) = \left[\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right]^T$$

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[-2.5,
?,
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
0.6,
?,
?,

$$(1.25353 - 1.25347)/0.0001 = 0.6$$

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?,...]

$$[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33, \dots]$$

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
0,
?,
?]

$$(1.25347 - 1.25347)/0.0001 = 0$$

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w+h) - L(w)}{h}$$

?

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
0,
?,
?,
?,
?,
?,
?,...]

Numeric Gradient

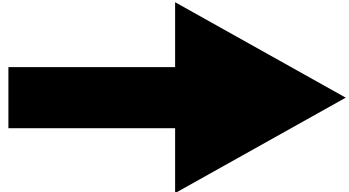
- Slow! Need to loop over all dimensions
- Approximate

This is silly. The loss is just a function of W :

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

$$L_i = -\log\left(\frac{e^{s y_i}}{\sum_j e^{s_j}}\right)$$

$$s = f(x; W) = Wx$$

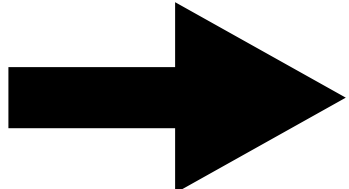
want $\nabla_W L$  $W = W - \eta \nabla_W L$

This is silly. The loss is just a function of W :

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

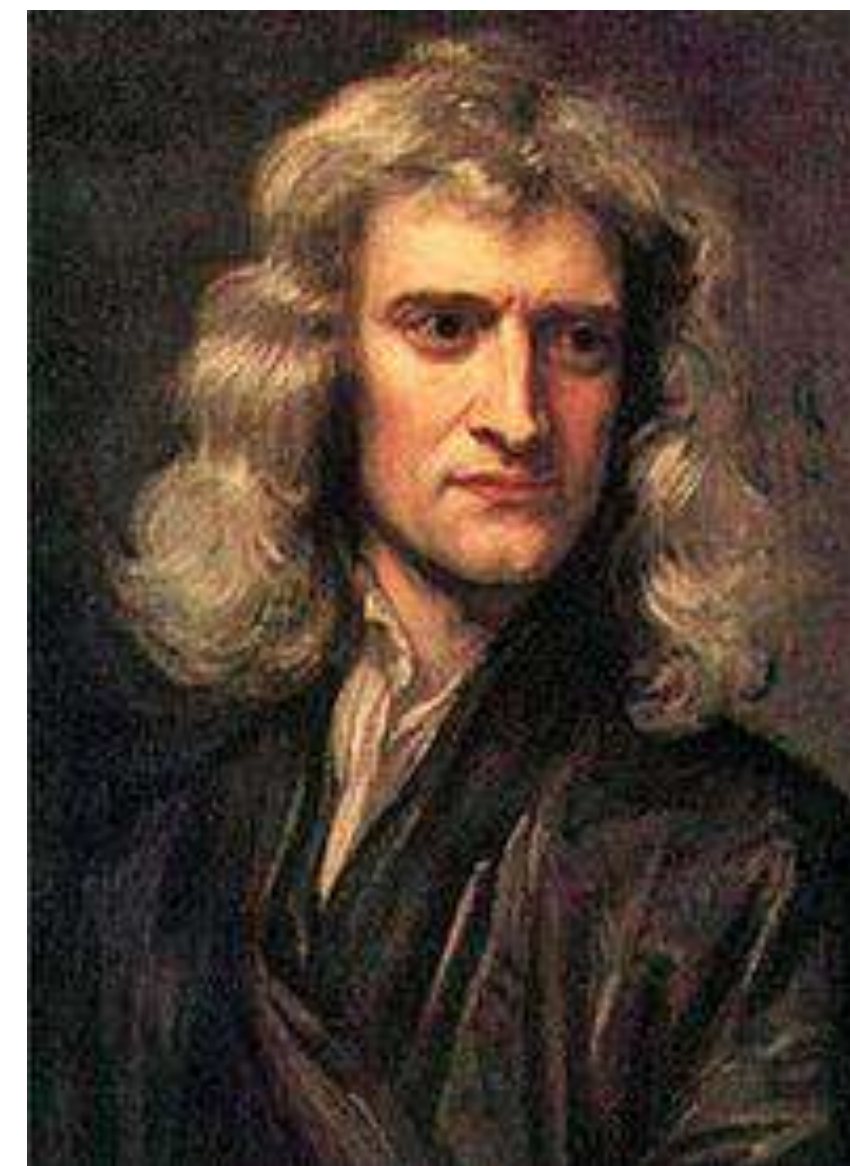
$$L_i = -\log\left(\frac{e^{s y_i}}{\sum_j e^{s_j}}\right)$$

$$s = f(x; W) = Wx$$

want $\nabla_W L$ 

$$W = W - \eta \nabla_W L$$

Use calculus to compute an
analytic gradient



Lets see how well Gradient Descent works on the toy example ...

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

$$\frac{\partial L(w_1, w_2)}{\partial w_1} = ?$$

$$\frac{\partial L(w_1, w_2)}{\partial w_2} = ?$$

Lets see how well Gradient Descent works on the toy example ...

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

$$\frac{\partial L(w_1, w_2)}{\partial w_1} = 2w_1 - 2$$

$$\frac{\partial L(w_1, w_2)}{\partial w_2} = w_2 + 1$$

```
def loss_fn(w):  
    """Compute the toy loss for a 2D parameter vector."""  
    w1, w2 = w  
    return (w1 - 2.0) ** 2 + 0.5 * (w2 + 1.0) ** 2  
  
def grad_fn(w):  
    """Compute the gradient of the toy loss."""  
    w1, w2 = w  
    d_w1 = 2.0 * (w1 - 2.0)  
    d_w2 = w2 + 1.0  
    return np.array([d_w1, d_w2])
```

Lets see how well Gradient Descent works on the toy example ...

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

```
def gradient_descent(start_w, learning_rate=0.2, num_steps=25):
    w = start_w.astype(float).copy()
    history = []

    for step in range(num_steps):
        loss = loss_fn(w)
        grad = grad_fn(w)
        history.append(
            {
                "step": step,
                "w": w.copy(),
                "loss": loss,
                "grad": grad.copy(),
            }
        )
        w = w - learning_rate * grad
    final_loss = loss_fn(w)
    return w, final_loss, history
```

Weight update

```
def loss_fn(w):
    """Compute the toy loss for a 2D parameter vector."""
    w1, w2 = w
    return (w1 - 2.0) ** 2 + 0.5 * (w2 + 1.0) ** 2

def grad_fn(w):
    """Compute the gradient of the toy loss."""
    w1, w2 = w
    d_w1 = 2.0 * (w1 - 2.0)
    d_w2 = w2 + 1.0
    return np.array([d_w1, d_w2])
```

Lets see how well Gradient Descent works on the toy example ...

We use a 2D parameter vector $w = [w_1, w_2]$, and define the loss as:

$$L(w_1, w_2) = (w_1 - 2)^2 + 0.5(w_2 + 1)^2$$

Gradient descent result

start_w = [-5. 4.]

best_w = [1.9999801 -0.98111053]

best_loss = 0.000178

step 0: w = [-5.00, 4.00], loss = 61.5000, grad = [-14.00, 5.00]

step 1: w = [-2.20, 3.00], loss = 25.6400, grad = [-8.40, 4.00]

step 2: w = [-0.52, 2.20], loss = 11.4704, grad = [-5.04, 3.20]

step 3: w = [0.49, 1.56], loss = 5.5629, grad = [-3.02, 2.56]

step 4: w = [1.09, 1.05], loss = 2.9202, grad = [-1.81, 2.05]

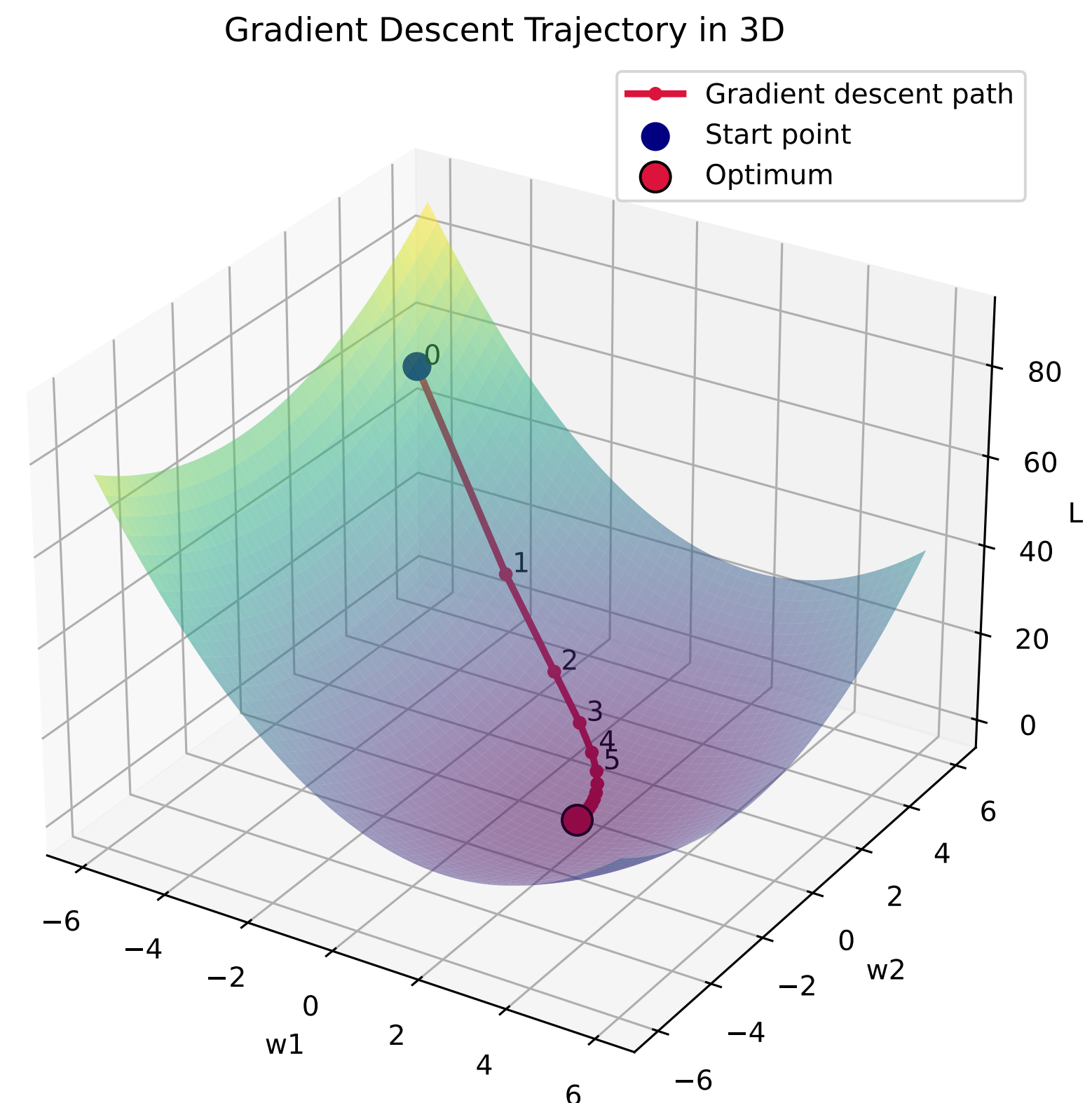
step 5: w = [1.46, 0.64], loss = 1.6385, grad = [-1.09, 1.64]

step 6: w = [1.67, 0.31], loss = 0.9657, grad = [-0.65, 1.31]

step 7: w = [1.80, 0.05], loss = 0.5882, grad = [-0.39, 1.05]

step 8: w = [1.88, -0.16], loss = 0.3657, grad = [-0.24, 0.84]

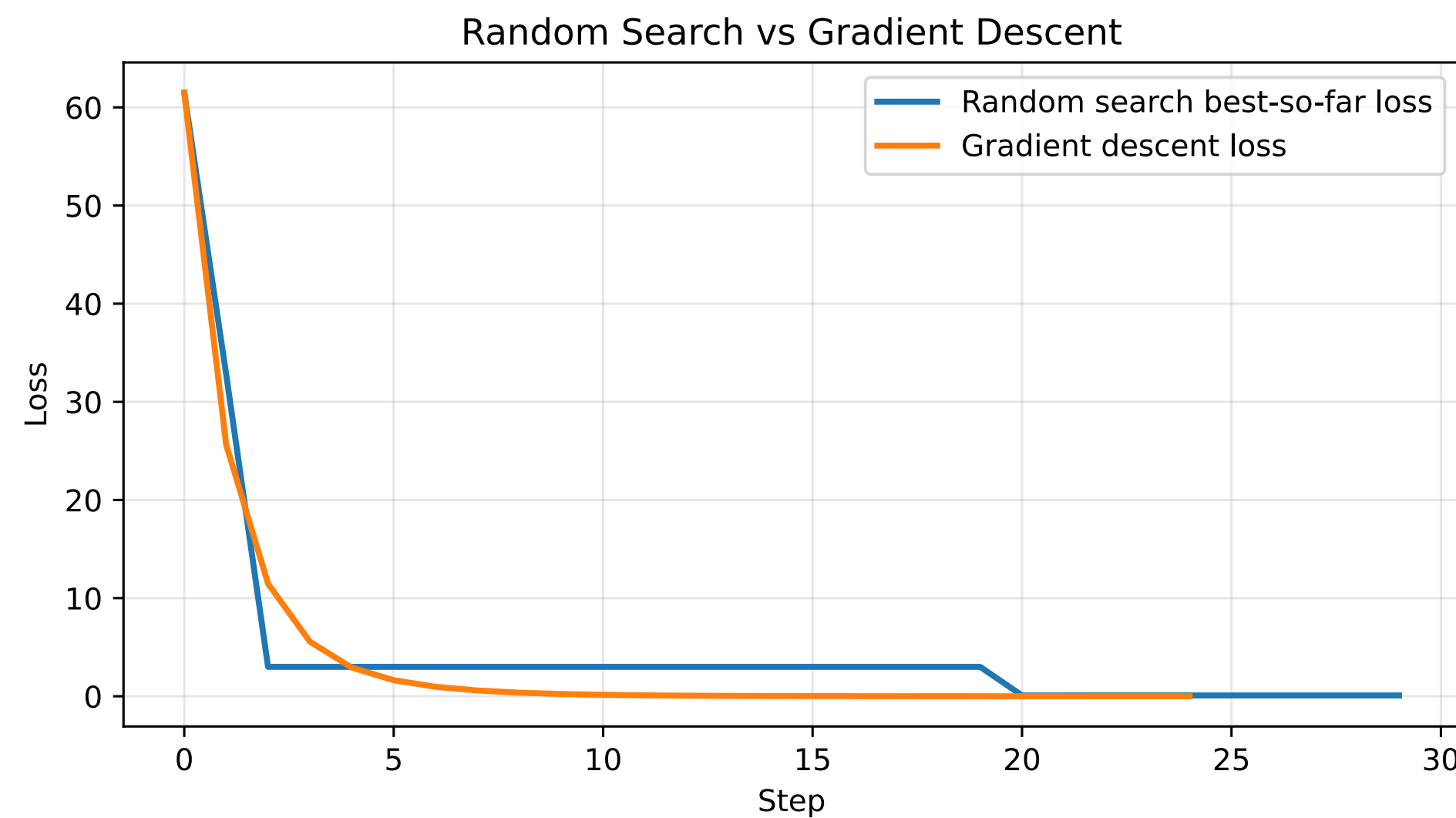
step 9: w = [1.93, -0.33], loss = 0.2302, grad = [-0.14, 0.67]



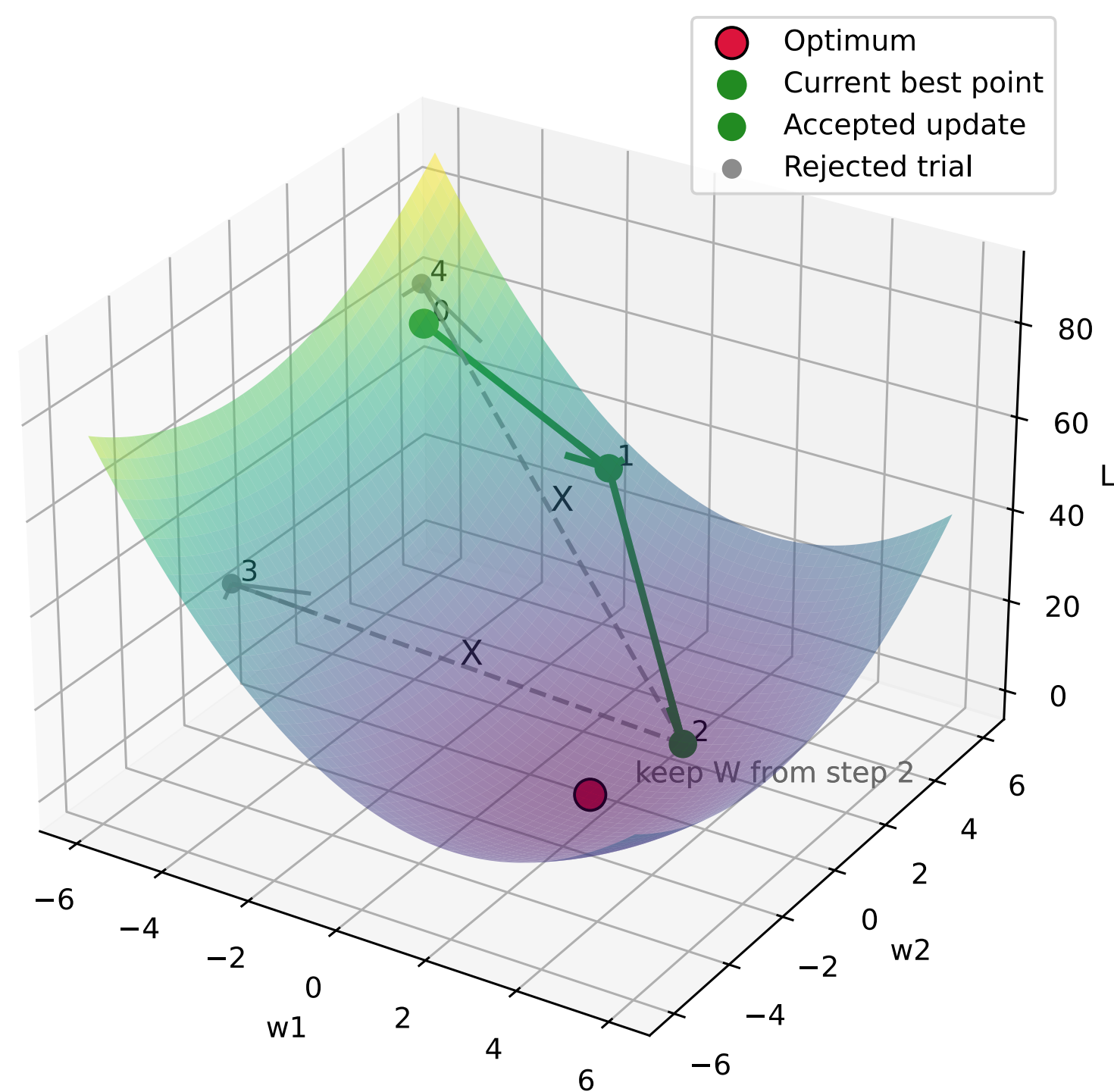
Random Search

V.S.

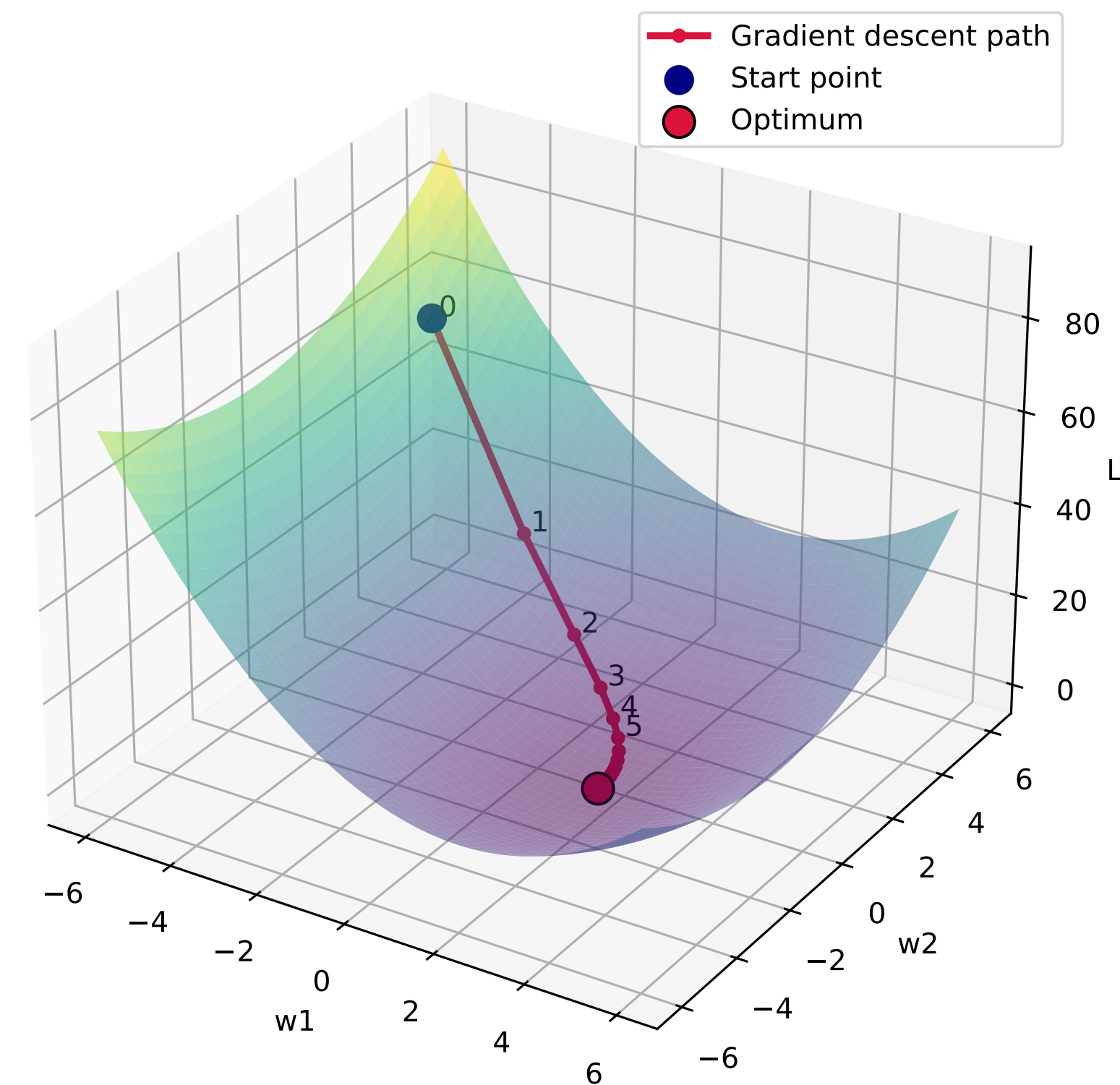
Gradient Descent



Random Search in 3D: Accepted Updates vs Rejected Trials



Gradient Descent Trajectory in 3D



Gradient Descent

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive
when N is large!

Approximate sum
using a **minibatch** of
examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent
```

```
while True:
```

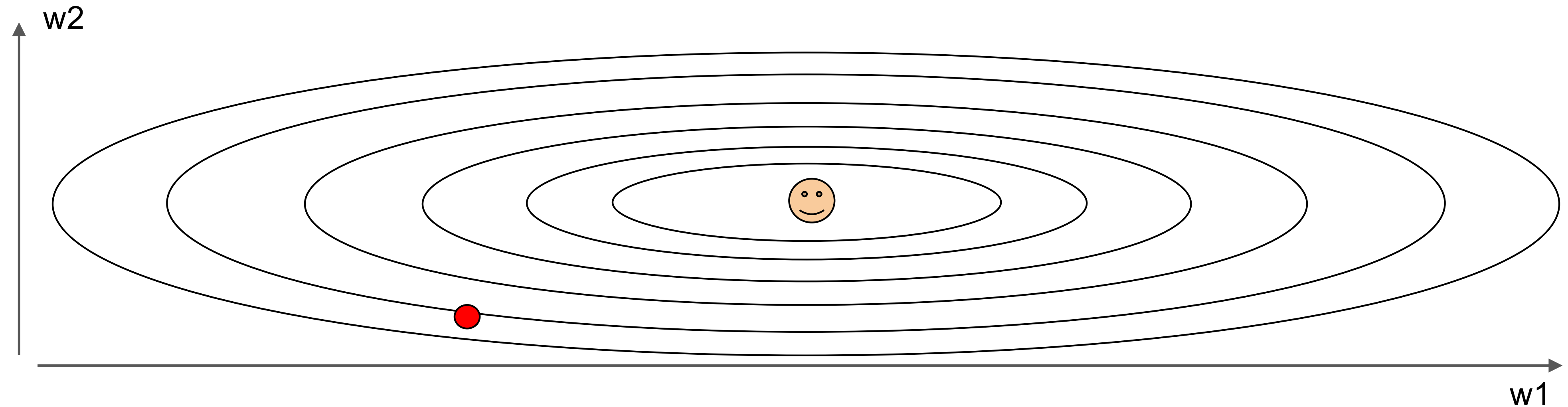
```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?

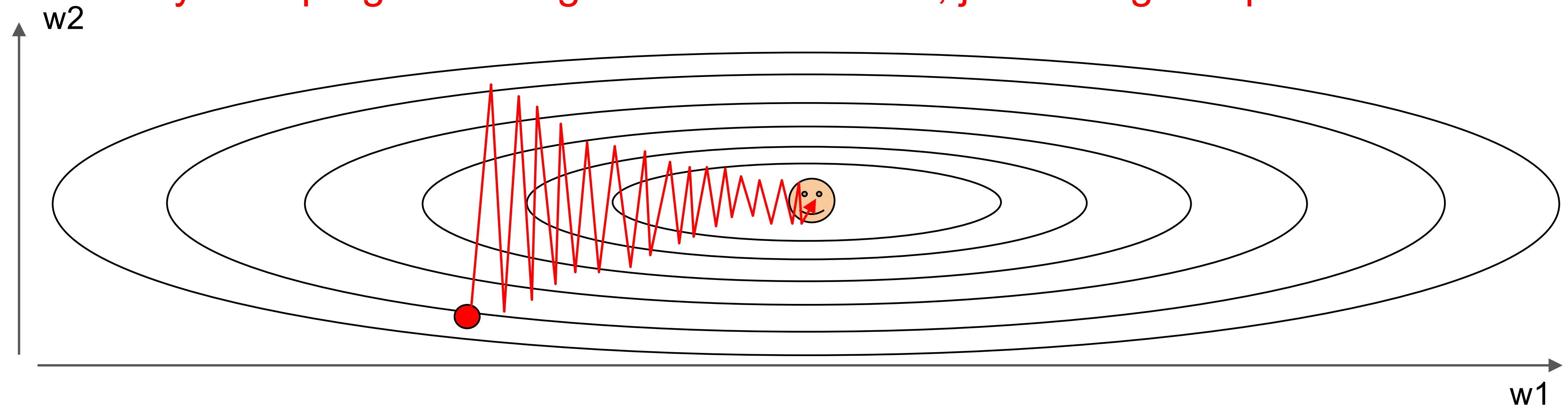


Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?

What does gradient descent do?

Very slow progress along shallow dimension, jitter along steep direction

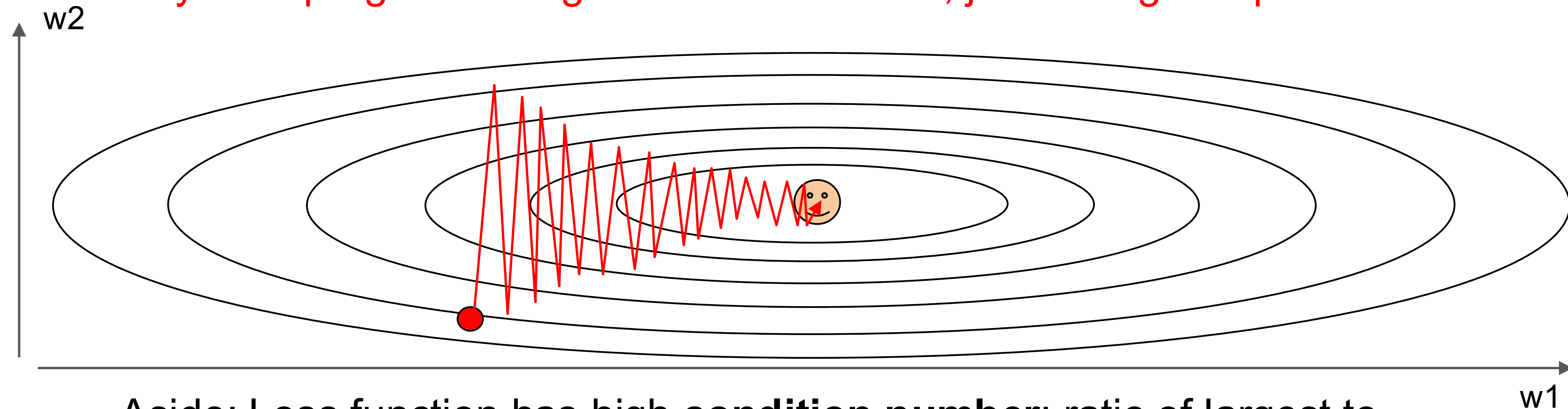


Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?

What does gradient descent do?

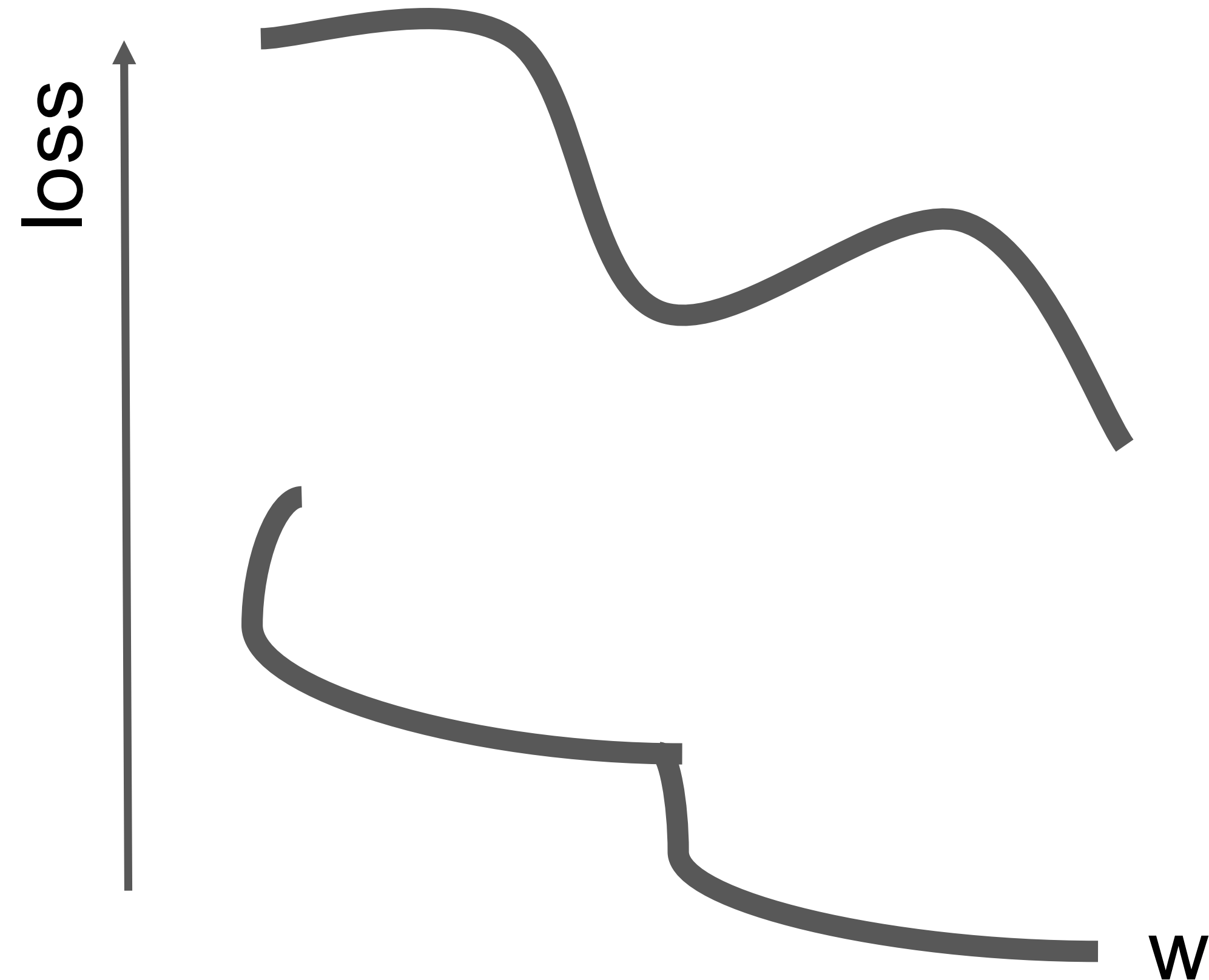
Very slow progress along shallow dimension, jitter along steep direction



Aside: Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

Optimization: Problem #2 with SGD

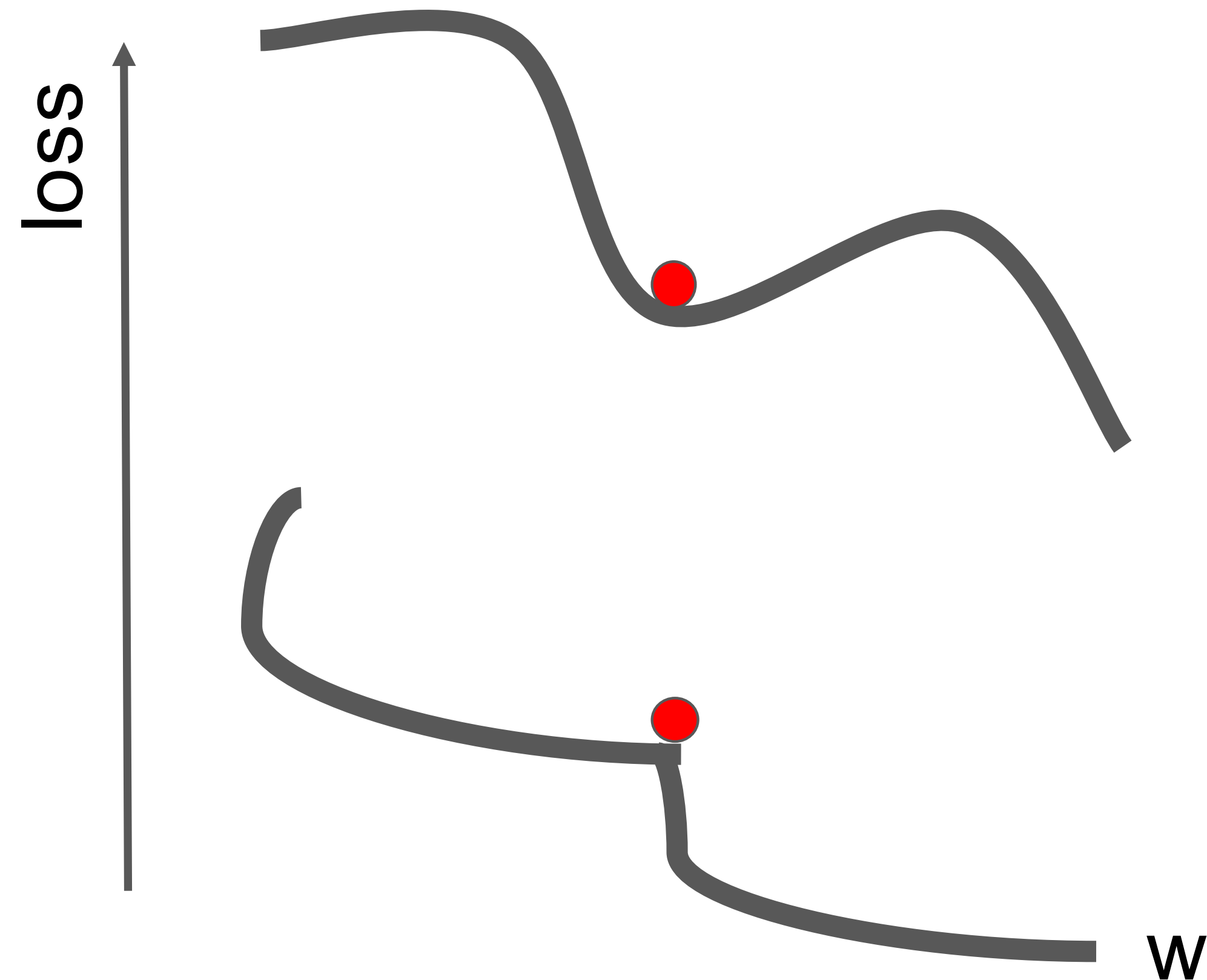
What if the loss function has a **local minima** or **saddle point**?



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

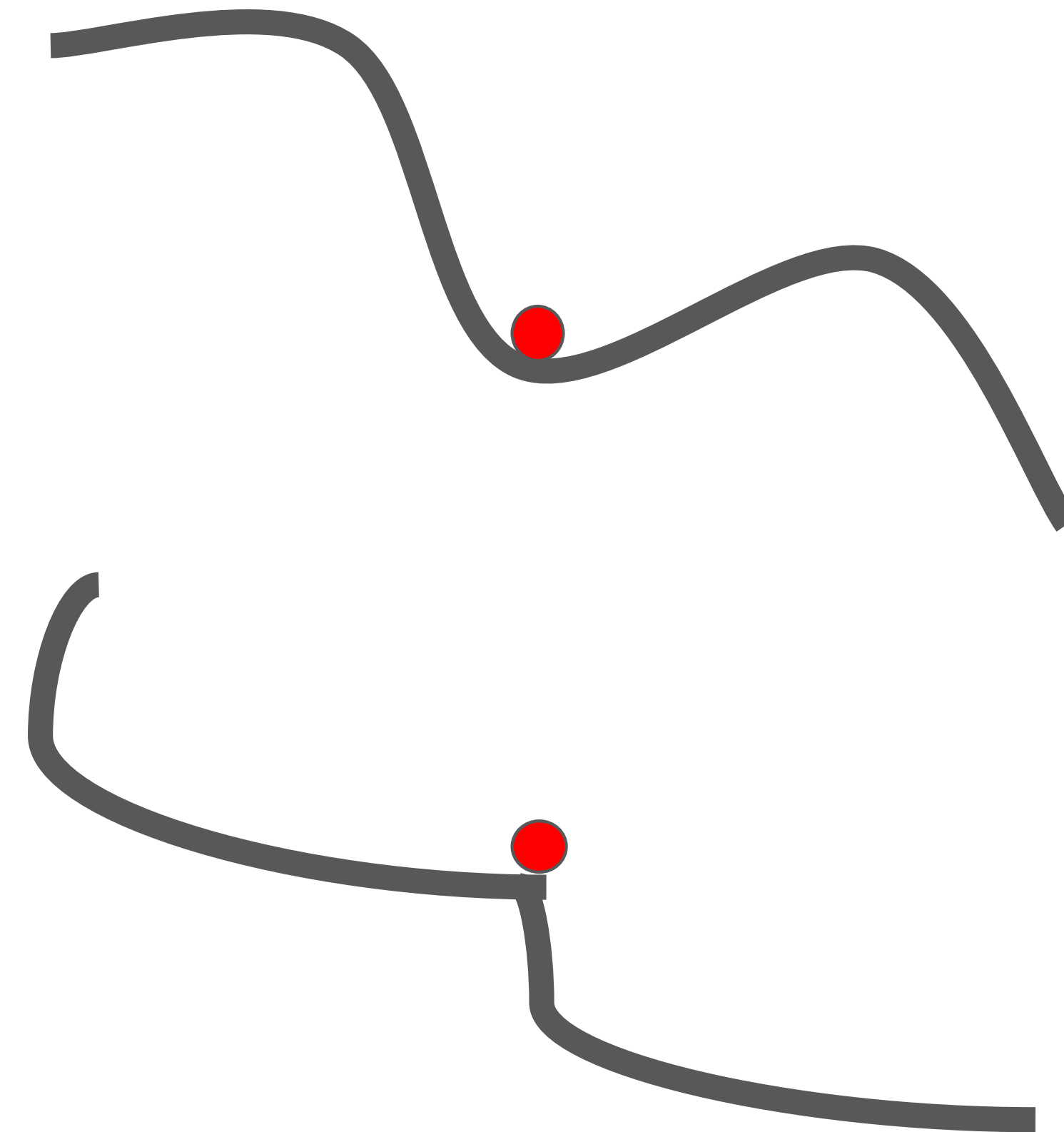
Zero gradient,
gradient descent
gets stuck



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension



Optimization: Problem #2 with SGD

saddle point in two dimension

$$f(x, y) = x^2 - y^2$$

$$\frac{\partial}{\partial x}(x^2 - y^2) = 2x \rightarrow 2(0) = 0$$

$$\frac{\partial}{\partial y}(x^2 - y^2) = -2y \rightarrow -2(0) = 0$$

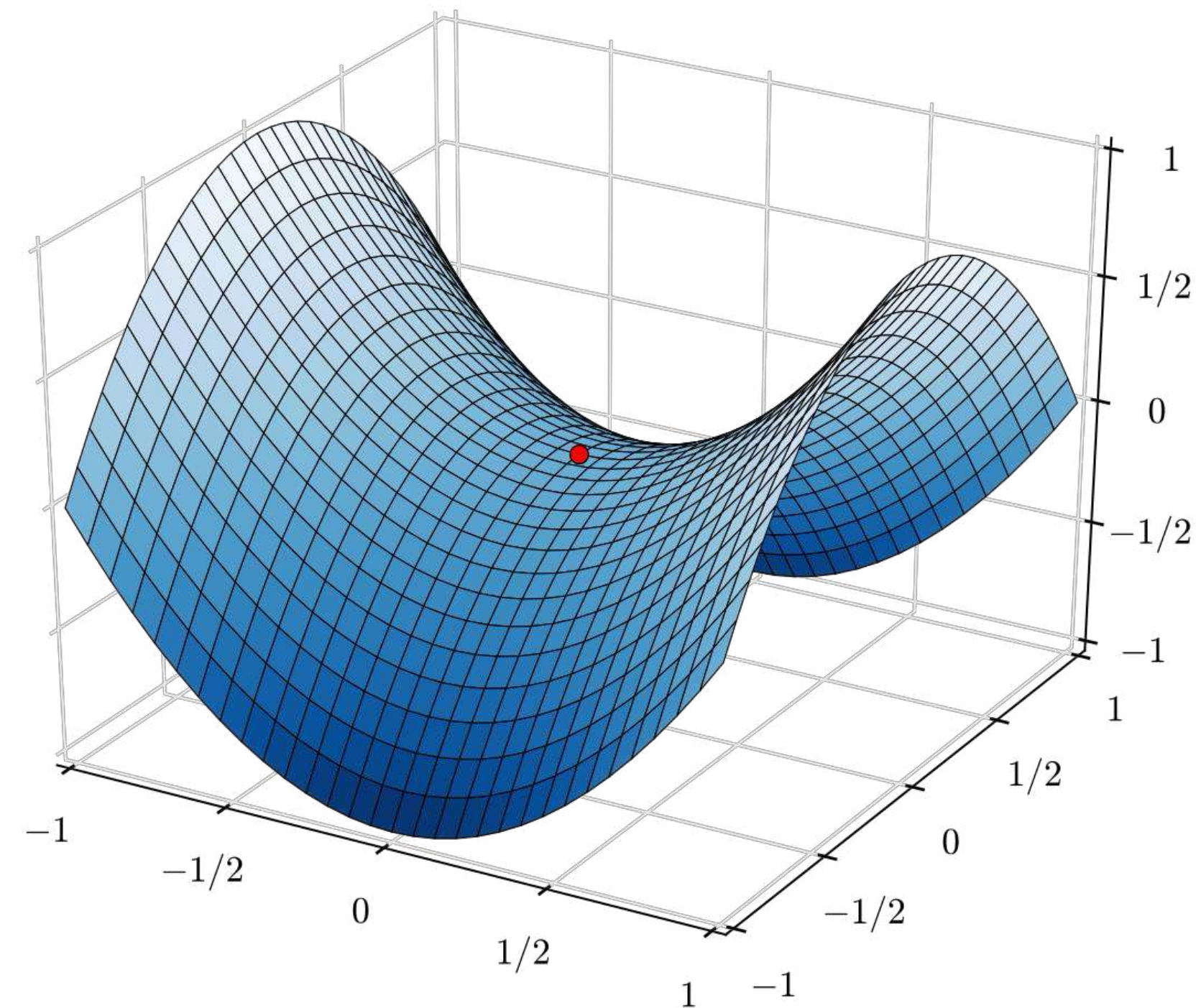


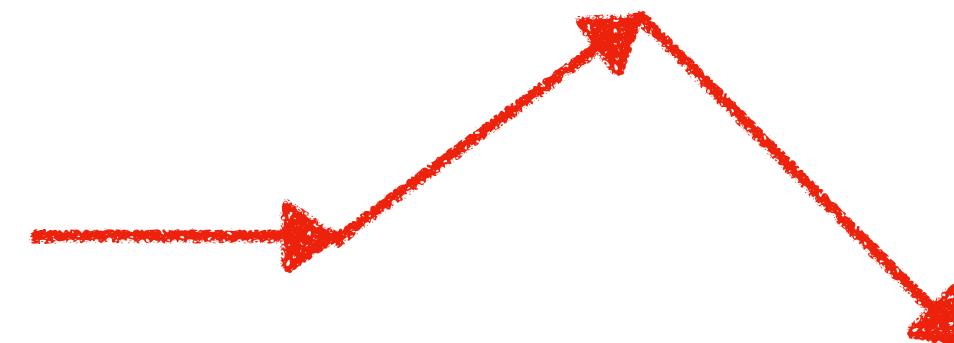
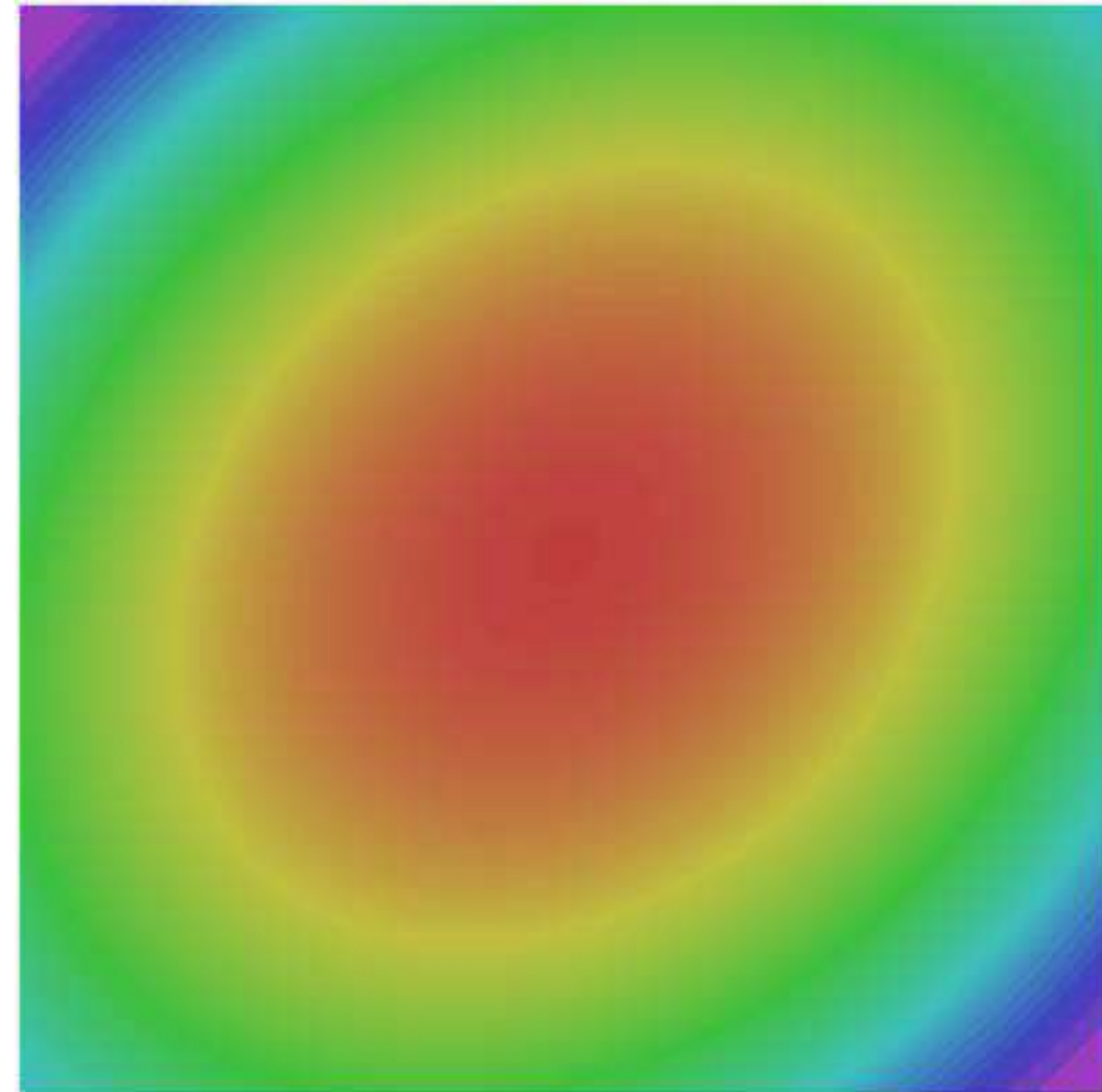
Image source: https://en.wikipedia.org/wiki/Saddle_point

Optimization: Problem #3 with SGD

Our gradients come from minibatches so they can be noisy!

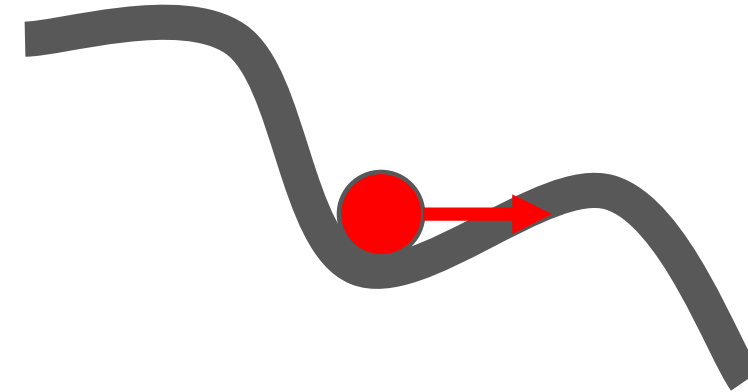
$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

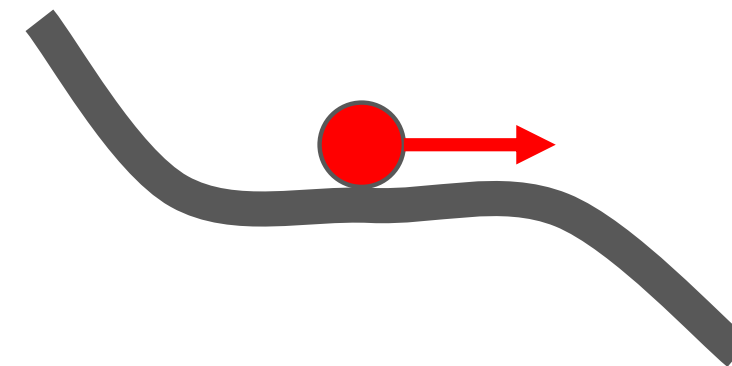


SGD + Momentum

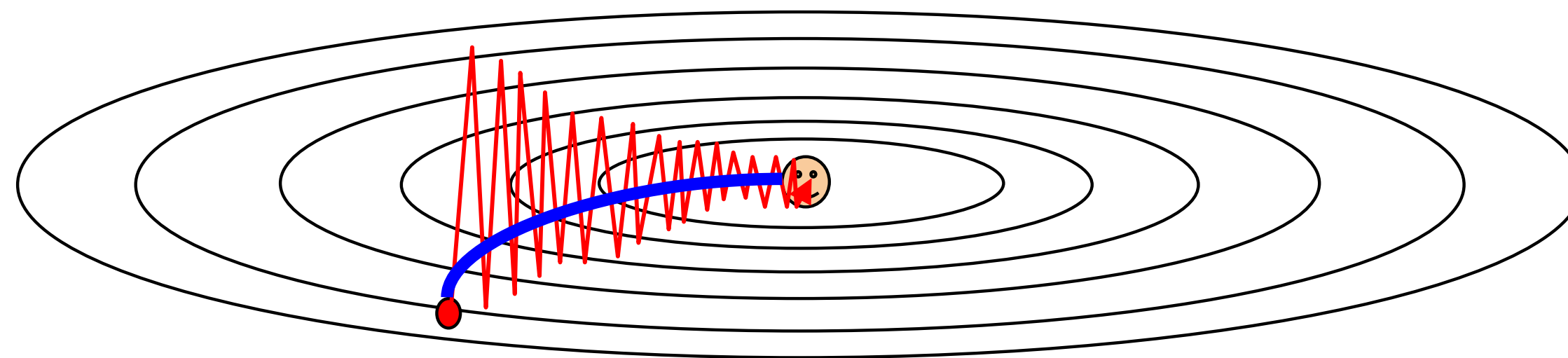
Local Minima



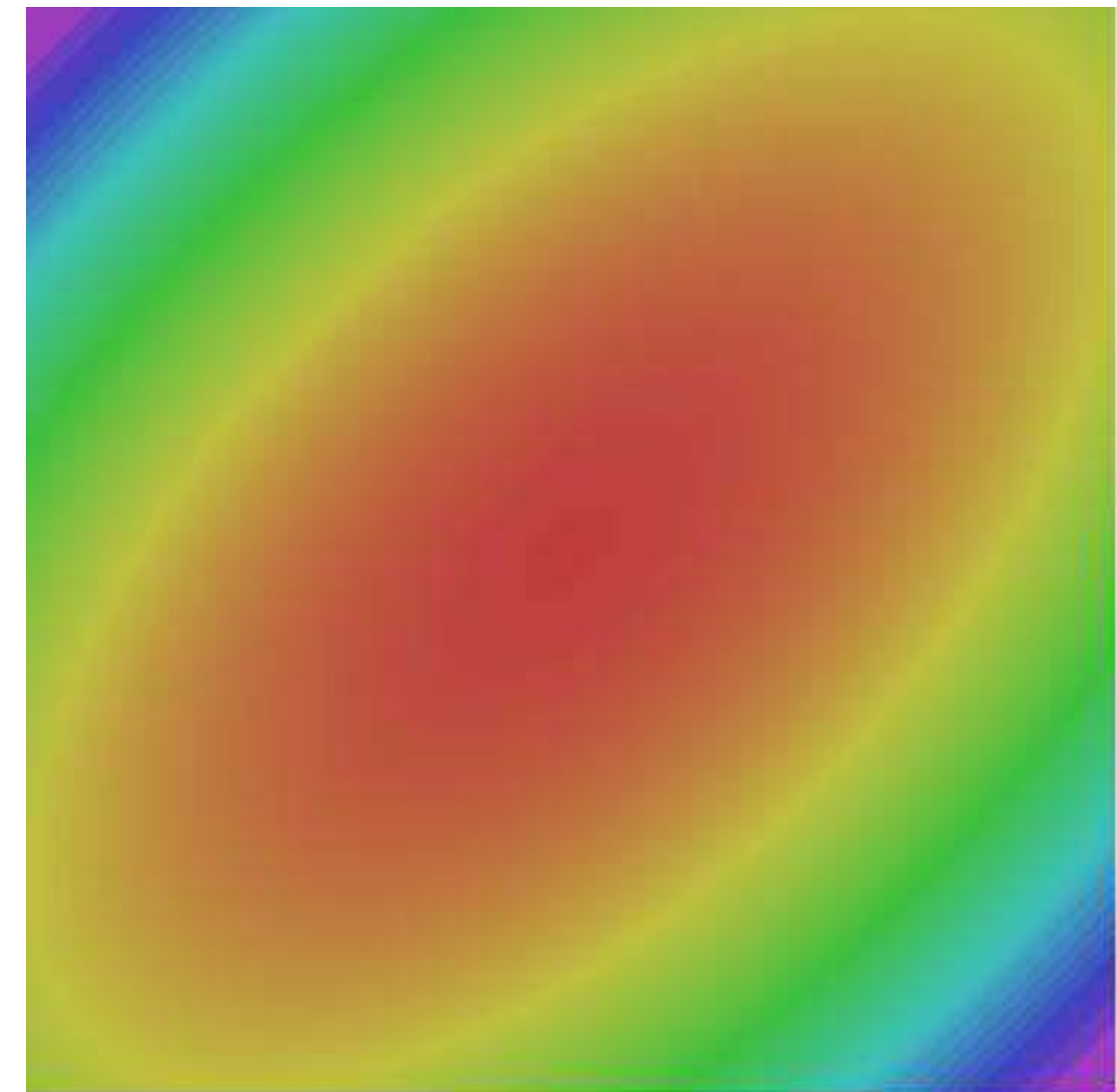
Saddle points



Poor Conditioning



Gradient Noise



SGD

SGD+Momentum

SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:  
    dx = compute_gradient(x)  
    x -= learning_rate * dx
```

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

- Build up “velocity” as a running mean of gradients
- Rho gives “momentum”; typically rho=0.9 or 0.99

Optimizers: Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Momentum

RMSProp

Sort of like RMSProp with momentum

Q: What happens at first timestep?

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that
first and second moment
estimates start at zero

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

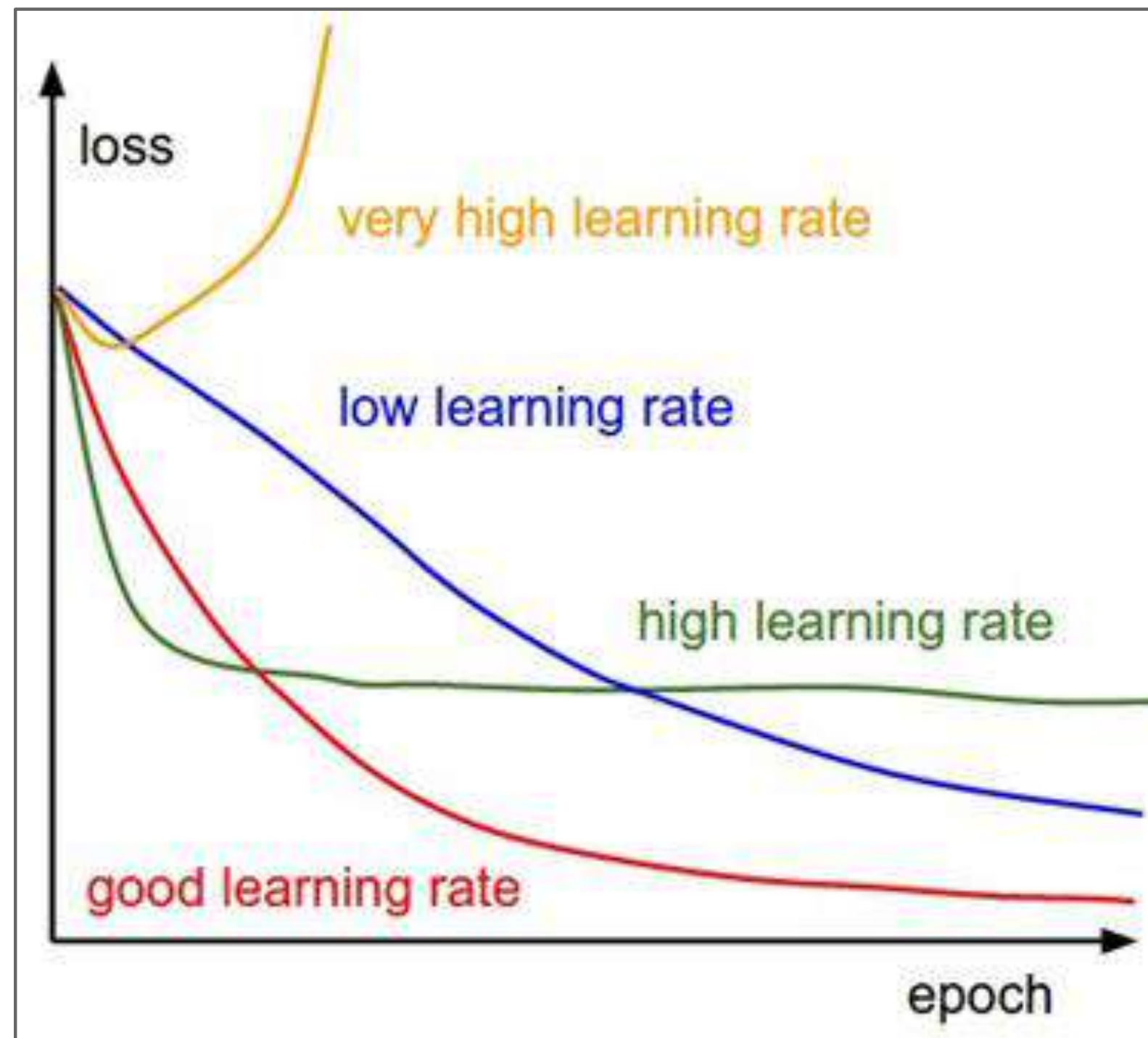
AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with $\text{beta1} = 0.9$, $\text{beta2} = 0.999$, and $\text{learning_rate} = 1\text{e-}3$ or $5\text{e-}4$ is a great starting point for many models!

Learning Rate

SGD, SGD+momentum, Adam all have **learning rate** as a hyperparameter. Which one of them is best to use?



SGD

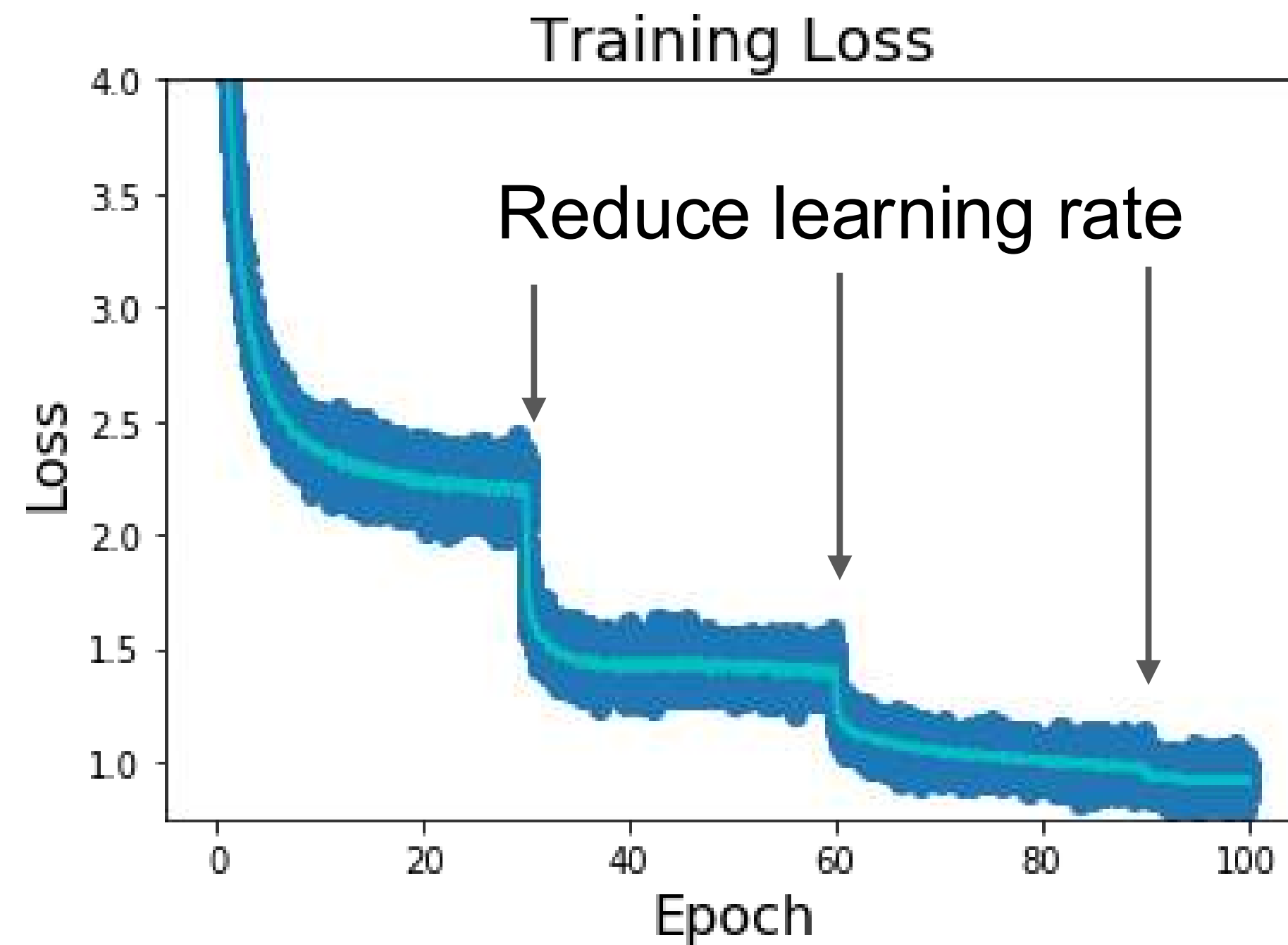
$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

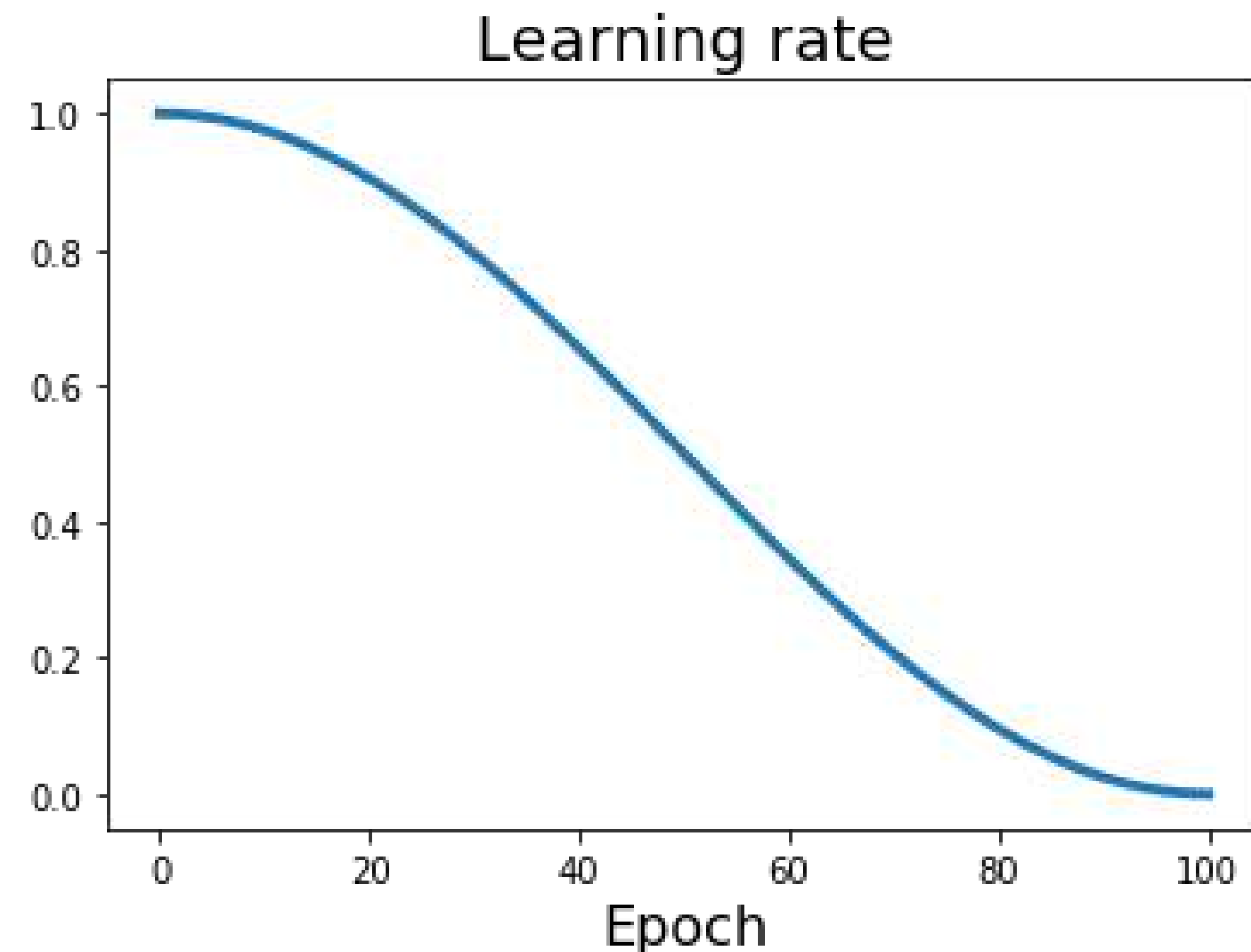
$$x_{t+1} = x_t - \alpha v_{t+1}$$

Learning rate decays over time



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

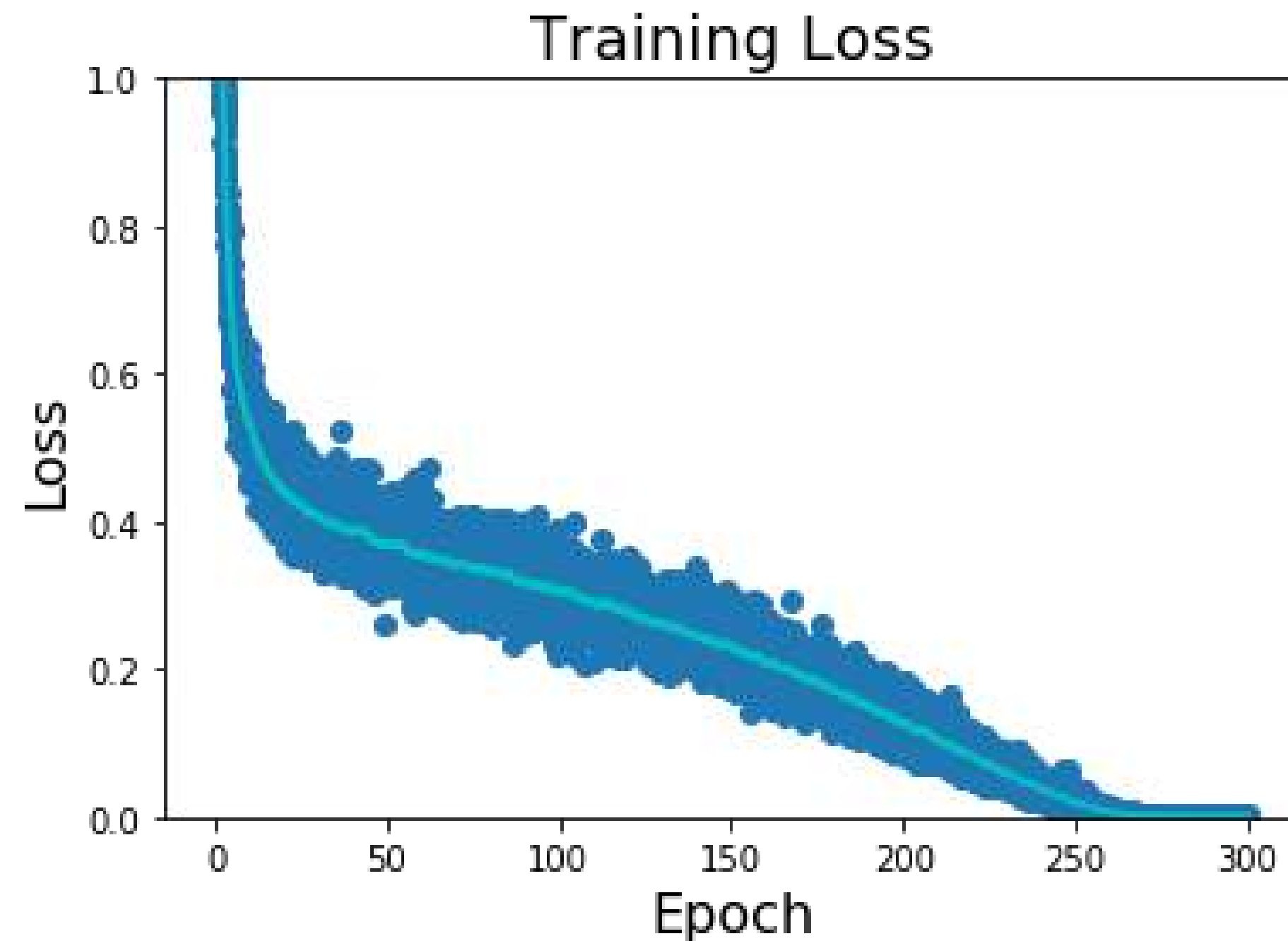
α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child et al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

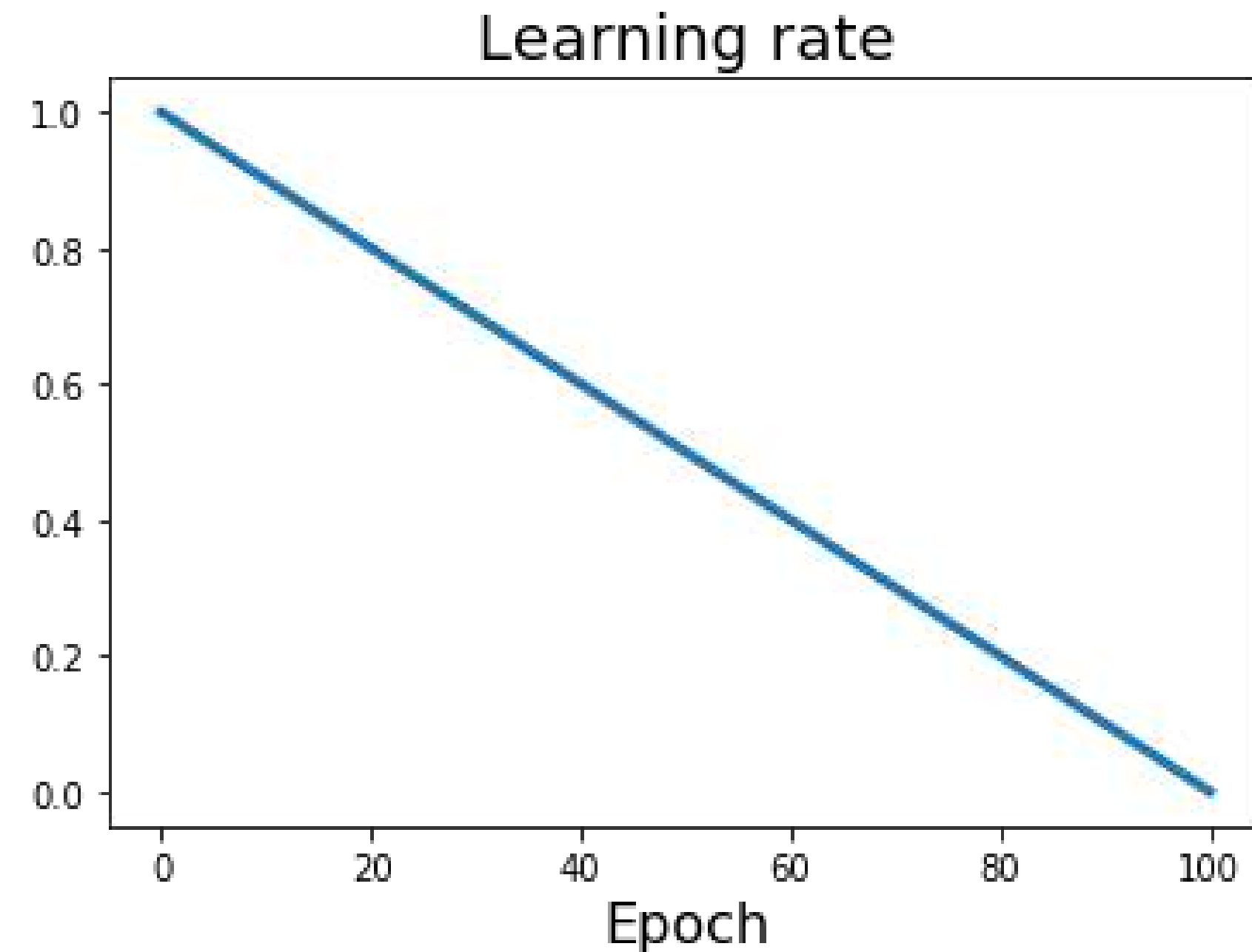
α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Loshchilov and Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts”, ICLR 2017
Radford et al, “Improving Language Understanding by Generative Pre-Training”, 2018
Feichtenhofer et al, “SlowFast Networks for Video Recognition”, arXiv 2018
Child et al, “Generating Long Sequences with Sparse Transformers”, arXiv 2019

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

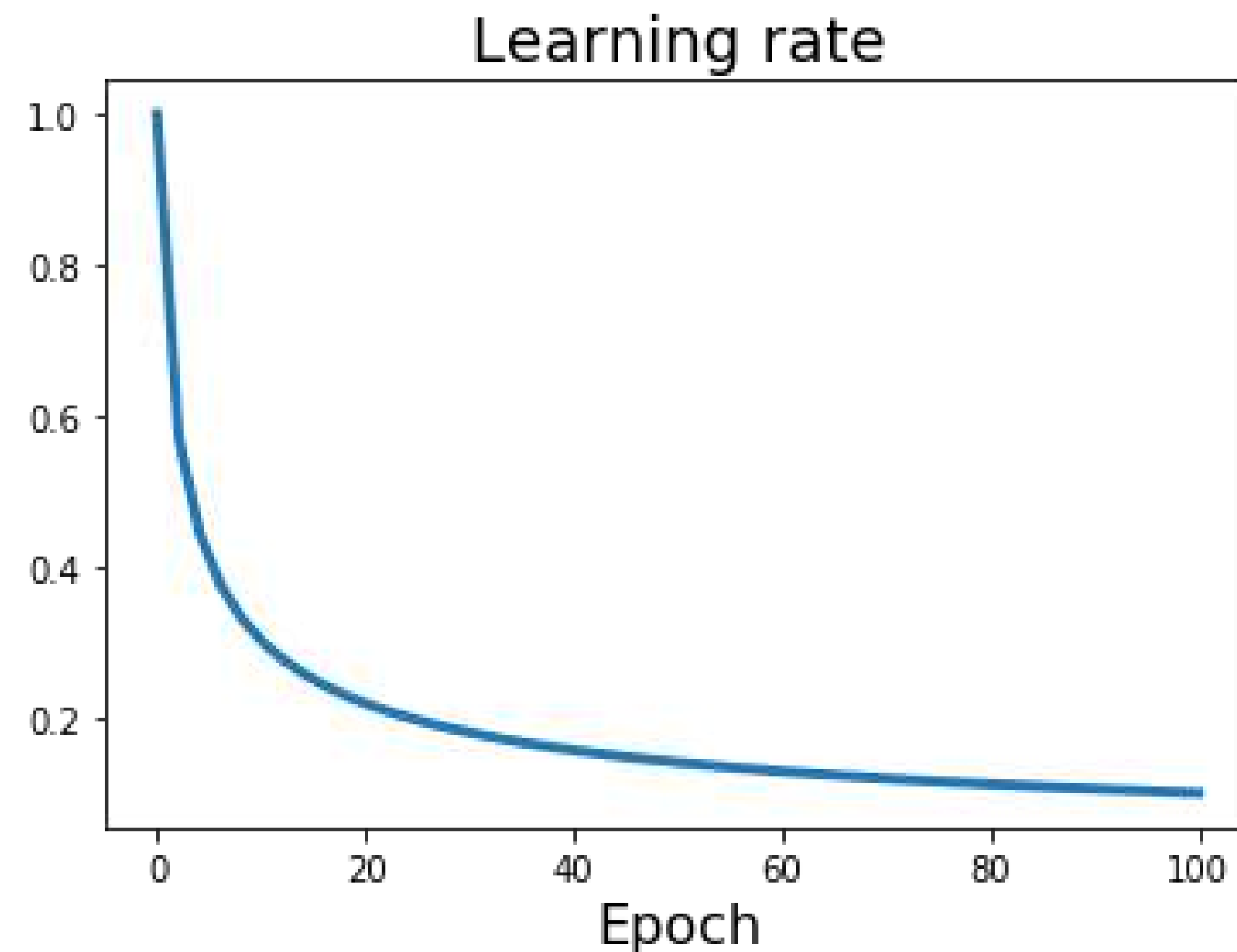
Linear: $\alpha_t = \alpha_0(1 - t/T)$

α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

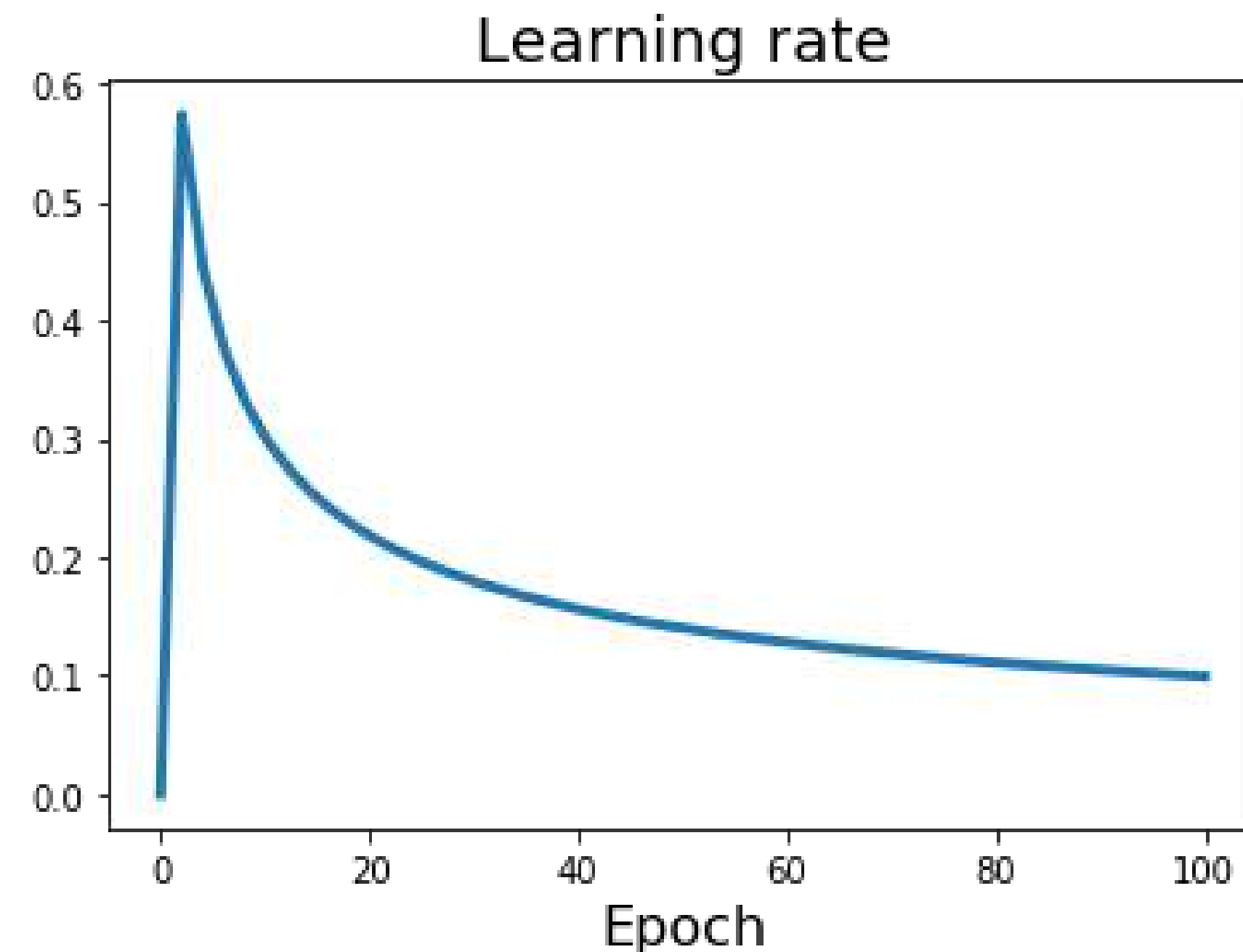
Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Learning Rate Decay: Linear Warmup



High initial learning rates can make loss explode; linearly increasing learning rate from 0 over the first ~5,000 iterations can prevent this.

Empirical rule of thumb: If you increase the batch size by N , also scale the initial learning rate by N

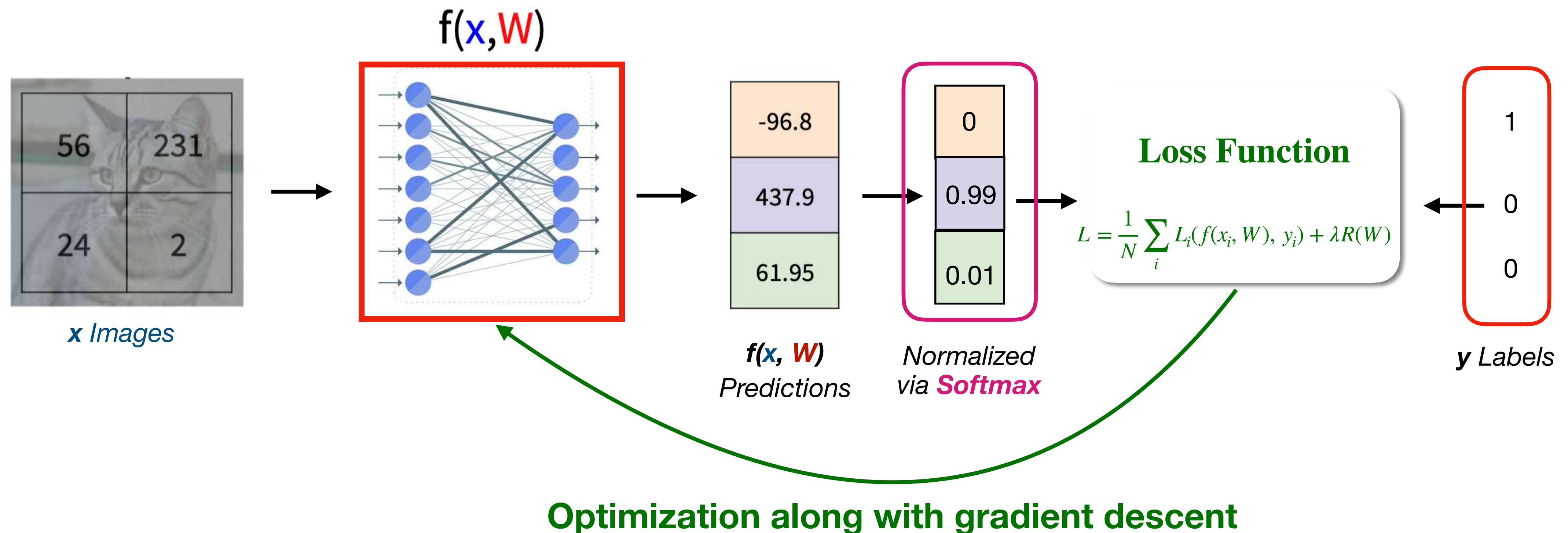
In practice:

- **Adam(W)** is a good default choice in many cases; it often works ok even with constant learning rate
- **SGD+Momentum** can outperform Adam but may require more tuning of LR and schedule

Linear Classifier

Model Training: Softmax Normalization + Cross-Entropy Loss + Regularization

Optimization: SGD + Momentum, AdamW, Learning Rate

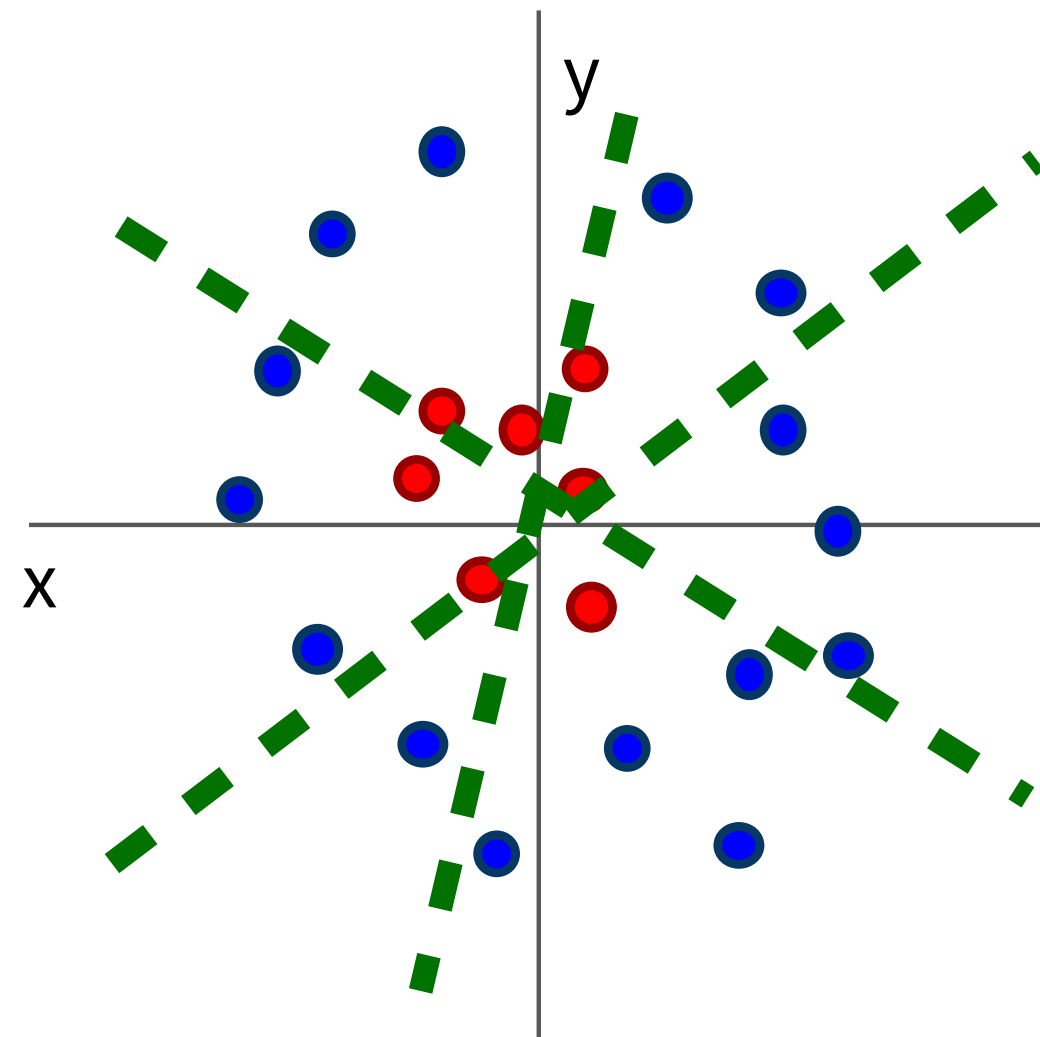


Looking Ahead: How to optimize more complex functions?

(Currently) Linear score function: $f = Wx$

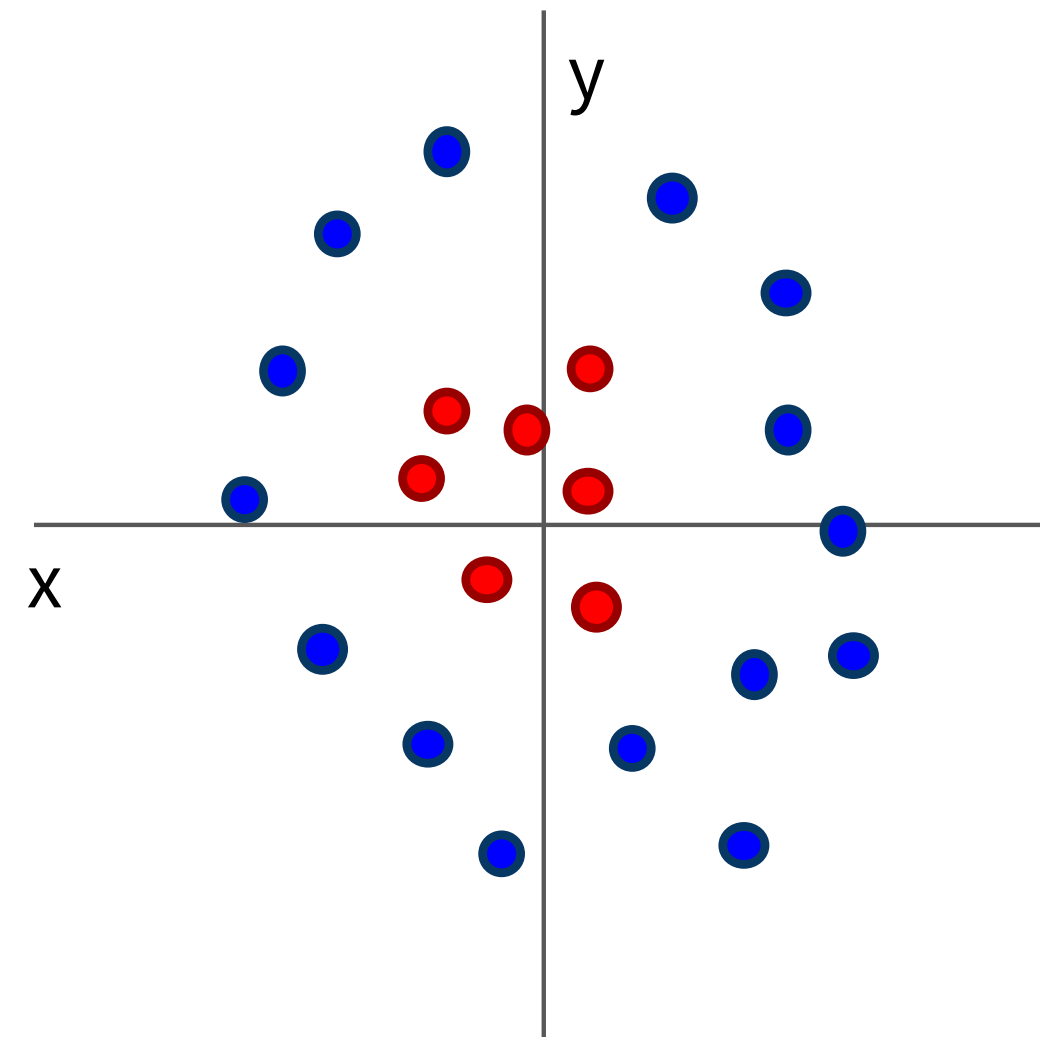
$$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$$

Why do we want non-linearity?



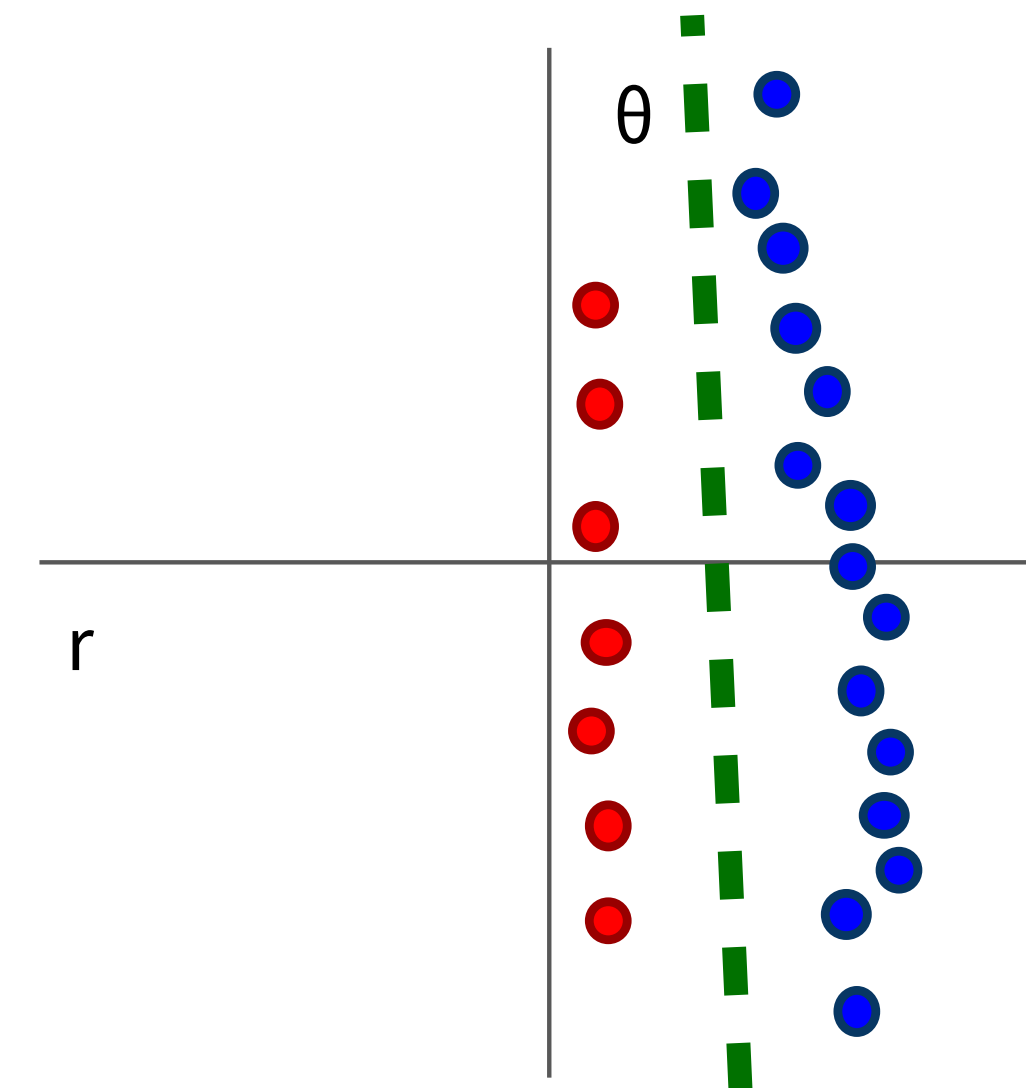
Cannot separate red and blue points with linear classifier

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$



After applying feature transform, points can be separated by linear classifier

Looking Ahead: How to optimize more complex functions?

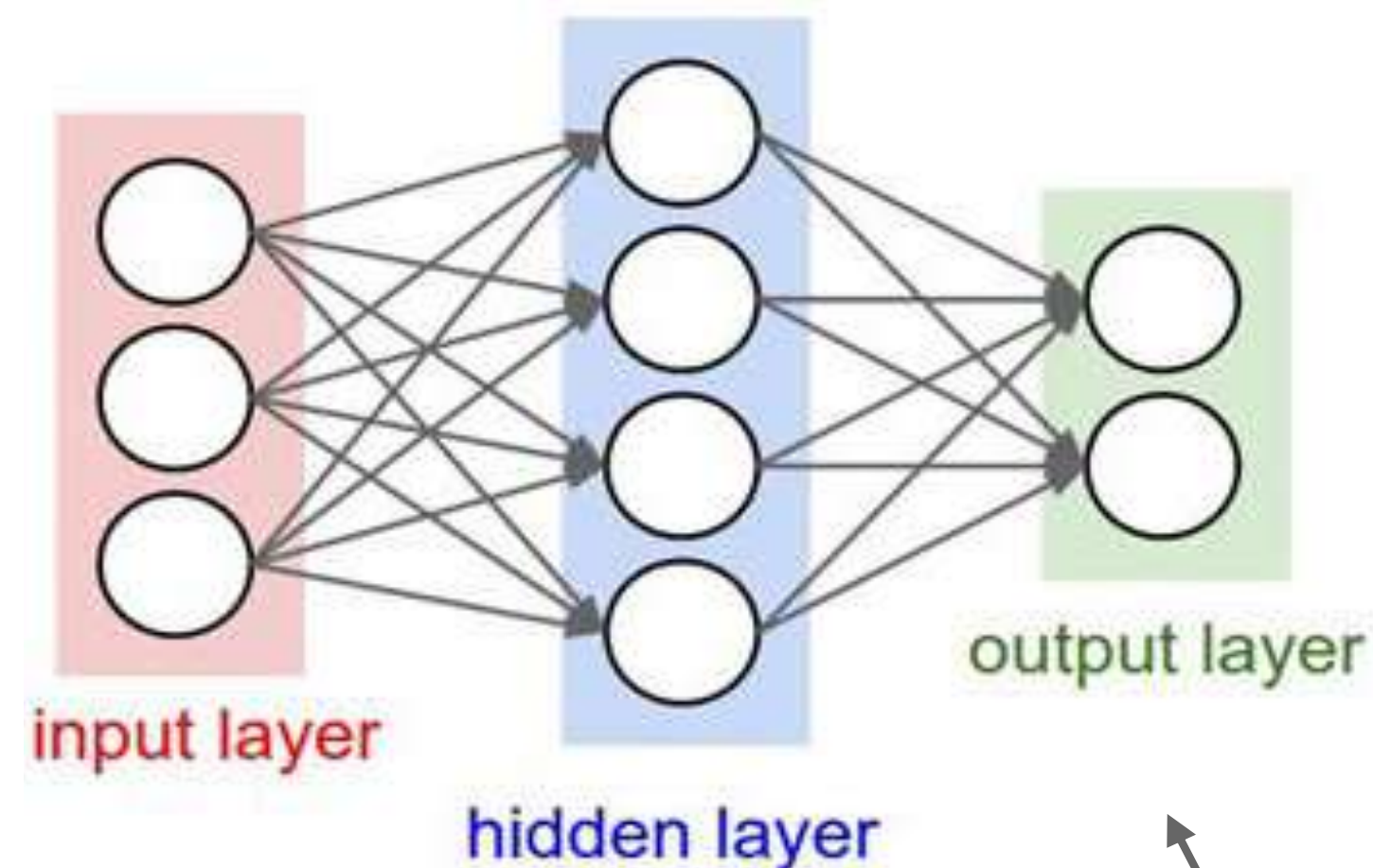
(Currently) Linear score function: $f = Wx$

$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$

2-layer Neural Network $f = W_2 (W_1 x)$

$$= (W_2 W_1) x$$

$$= Wx$$

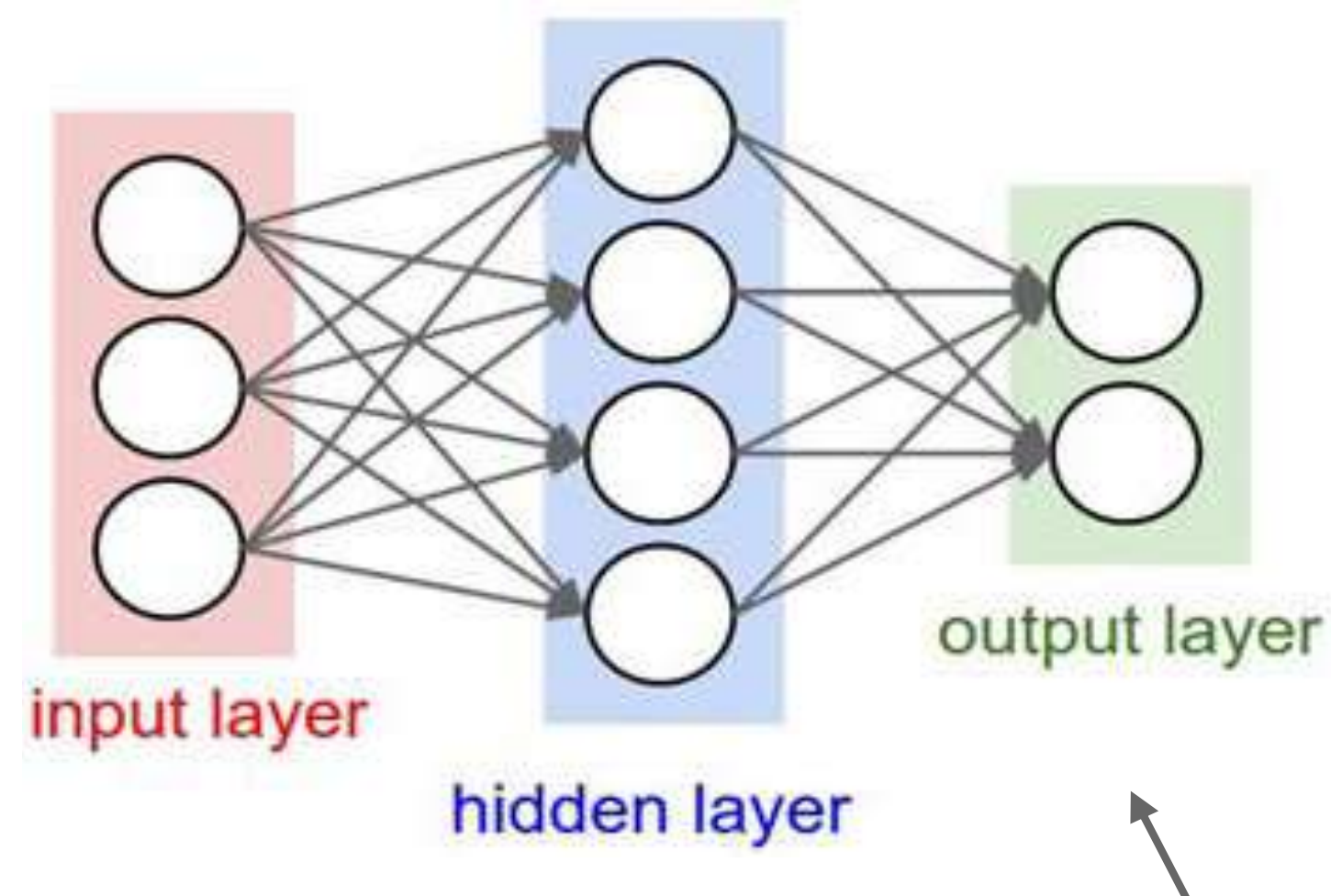


Neural networks: 2 layers

(Currently) Linear score function: $f = Wx$

(Next Class) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$ **Activation Function**

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$



(In practice we will usually add a learnable bias at each layer as well)

Neural networks: also called fully connected network

(Currently) Linear score function: $f = Wx$

(Next Class) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$ Activation Function

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

“Neural Network” is a very broad term; these are more accurately called “fully-connected networks” or sometimes “multi-layer perceptrons” (MLP)

(In practice we will usually add a learnable bias at each layer as well)

Next Lecture: Multi-layer Neural Net & Backpropagation

